

LAMBDA CALCUL, TYPAGE ET INFÉRENCE

catapillie

MY NEW LANGUAGE IS GREAT, BUT IT HAS A FEW QUIRKS REGARDING TYPE:

```
[1]> 2+"2"  
=> "4"  
[2]> "2"+[]  
=> "[2]"  
[3] (2/0)  
=> NaN  
[4] (2/0)+2  
=> NaP  
[5] <"" + "">  
=> "<+>"  
[6] [1,2,3]+2  
=> FALSE  
[7] [,2,3]+4  
=> TRUE  
[8] 2/(2-(3/2+1/2))  
=> NaN.00000000000000013  
[9] RANGE(" ")  
=> (ji n j n j q j n j q j i n j)  
[10] > +2  
=> 12  
[11] > 2+2  
=> DONE  
[14] > RANGE(1,5)  
=> (1,4,3,4,5)  
[13] > FLOOR(10.5)  
=> |  
=> |  
=> |  
=> |____|0.5____|
```

<https://xkcd.com/1537/>

Introduction

Presque tous les langages de programmation relativement récents possèdent le typage comme fonctionnalité essentielle. Cela peut être une des raisons pour laquelle on choisirait un langage plutôt qu'un autre.

En effet, un programme bien typé empêche toute sorte d'opération incohérente entre plusieurs valeurs. Si le langage en question applique le typage au programme *avant* la compilation (on dit qu'il est « statiquement typé »), alors ces erreurs sont détectées avant l'exécution, ce qui permet d'éviter de nombreux problèmes...

Cependant, le typage ne permet pas seulement d'empêcher les erreurs : il permet par exemple de rendre un langage plus expressif et plus permissif, par exemple grâce au *polymorphisme*. En connaissant le type d'une fonction, il est aussi plus facile de deviner ce qu'elle fait. Le typage sert donc comme outil de documentation.

Mais ces bénéfices ont un prix : il faut connaître le type de toutes les valeurs que l'on manipule avant l'exécution du programme. Dans certains langages de programmation possédant un système de types assez simple (comme le C), ce n'est pas vraiment un problème. Dans d'autres langages comme OCaml, qui donne accès au polymorphisme et donc à des combinaisons infinies de types, on serait bien embêté d'indiquer tous les types des étapes intermédiaires. On est même parfois obligé de le faire au risque que le compilateur/l'analyseur sémantique n'ait pas assez d'informations (voir les fichiers d'en-tête pour les modules en OCaml).

Au vu de ce problème, il existe des algorithmes permettant de retrouver les types des valeurs d'un programme sans que le programmeur n'ait à les écrire : on parle d'un algorithme **d'inférence de type**.

Dans cet exposé, on se propose de redécouvrir un algorithme d'inférence de type pour un système de type qui ressemble à celui d'OCaml : l'algorithme J. Pour cela, on étudiera d'abord un langage ultra-simplifié, le λ -calcul, auquel on ajoutera des types. On donnera ensuite des éléments de preuve pour montrer qu'un programme bien typé sera correctement évalué. Enfin, nous pourrons en déduire un algorithme d'inférence qui tente d'appliquer les règles de typage.

1 Le lambda-calcul

Pourquoi étudier un autre langage plutôt que celui d'OCaml ? Nous aurons par la suite besoin d'une manière de parler de l'évaluation d'un programme, et pour cela, le langage OCaml est trop gros. Une fois que l'on aura prouvé nos propriétés sur les programmes bien typés, il sera en théorie très simple d'étendre les preuves sur toutes les constructions du langage OCaml.

Mais pour le moment, on décide de se réduire à un langage simpliste qui permet néanmoins toutes les opérations que peut exécuter OCaml. D'où l'intérêt de choisir un langage Turing-complet.

Un langage qui fonctionne parfaitement pour cette étude est le **lambda-calcul** (aussi noté λ -calcul), introduit dans les années 1930 par Alonzo Church. Dans les années 1940, il introduit une nouvelle version avec le typage : c'est le **lambda-calcul simplement typé**, aussi noté $\lambda^\rightarrow$.

1.1 Initiation au lambda-calcul et définitions

Le lambda-calcul est un mini-langage dont les seules primitives sont les fonctions et les variables. On définit une manière d'écrire les fonctions, les variables et les applications de fonctions dans ce langage de manière condensée.

Définition On définit Λ , le langage des λ -termes à l'aide d'une grammaire non-contextuelle.

On prendra comme non-terminaux : $\mathcal{V} = \{E, X\}$. Notre alphabet sera constitué de deux symboles uniques ' λ ' et '.' réservés pour la syntaxe, auquel on ajoute tous les noms de variables que l'on souhaite.

$$\Sigma = \{\lambda, .\} \cup \{a, b, \dots, x, y, \dots\}$$

On désigne par E le symbole initial, et les règles de production sont alors :

$$\begin{array}{ll} E \rightarrow X & \text{(variable)} \\ \rightarrow \lambda X. E & \text{(abstraction)} \\ \rightarrow E E & \text{(application)} \\ X \rightarrow a \mid b \mid \dots \mid x \mid y \mid \dots \end{array}$$

Remarque Une syntaxe plus concise pour la définition du langage des λ -termes est :

$$e := x \mid \lambda x. e \mid e e$$

Au passage, ceci définit bien l'ensemble des λ -termes comme un ensemble inductif.

Les λ -termes sont seules expressions autorisée dans le langage. Mais à quoi correspondent-elles ? Comme l'indiquent les annotations à côté de chaque règle de dérivation, un λ -terme décrit soit une variable x , soit une application d'une fonction f à un autre terme e , soit une fonction elle-même, que l'on note $\lambda x. e$ (où x est l'argument).

Ainsi, lorsque l'on voit $\lambda x. e$, on peut le lire « la fonction qui à x associe e ».
En pratique, voilà ce que cela donne pour quelques fonctions usuelles.

Exemple Prenons la fonction identité $x \mapsto x$. Le λ -terme correspondant serait :

$$\lambda x. x$$

Et ensuite, si on voulait l'appliquer à y , on écrirait :

$$(\lambda x. x) y$$

Exemple Un autre exemple, partons cette fois-ci d'une fonction en OCaml.

```
let f x y = x
```

En OCaml, les fonctions ne prennent qu'un seul argument, mais on peut "émuler" la fonctionnalité de plusieurs arguments avec la curryfication :

```
let f = fun x -> fun y -> x
```

Le λ -terme équivalent est alors :

$$\lambda x. \lambda y. x$$

Que ce cache-t-il derrière les variables x, y , etc ? Rien, ce sont des lambda-termes également, et deviendront soit des applications soit des fonctions lorsqu'on les évaluera.

Définition On notera dans toute la suite la fonction identité avec :

$$\text{id} = \lambda x. x$$

1.2 α -équivalence

Si vous vous souvenez bien de votre cours sur la logique du premier ordre, les variables pouvaient être liées par un quantificateur $\forall$ ou $\exists$, et on disait qu'une telle variable était *liée*, ou sinon, on disait qu'elle était *libre*.

$$\exists x \mid \forall y, (R(x) \implies R(y))$$

Il en va de même pour les λ -termes.

Définition On dit qu'une variable x d'un λ -terme est *liée* si elle fait partie d'un terme de la forme $(\lambda x. \dots)$:

$$(\lambda a. \dots (\lambda x. \dots x \dots)) \dots$$

On dit que le $\lambda x.$ le plus proche de x est son *lieur*.

Une variable est *libre* dans un terme si elle n'y est pas liée.

Enfin, pour $e \in \Lambda$, on peut noter $BV(e)$ l'ensemble des liées dans e , et $FV(e)$ l'ensemble des variables libres dans e .

Qui dit « variable libre » dit renommage : on peut renommer les variables dans un λ -terme sans que ce dernier ne « change de sens ». Par contre, si l'on essaie de renommer une variable qui est déjà liée dans un terme, on obtient quelque chose de différent. Pour le voir avec des fonctions, j'ai le droit de faire ça :

$$(x \mapsto f(x)) = (y \mapsto f(y))$$

mais pas ça, puisque je ne sais plus quelle variable est qui :

$$(x \mapsto f(x)) \neq (f \mapsto f(f))$$

Définition On dit que deux λ -termes e_1 et e_2 sont α -équivalents s'ils sont égaux à renommage près. J'emploierai la notation suivante :

$$e_1 \equiv_{\alpha} e_2$$

On peut dire que l'on travaille *module α -équivalence*, car le nom des variables n'est pas important lorsque l'on effectue une opération sur les lambda-termes (on peut toujours renommer en un terme convenable).

Théorème La relation d' α -équivalence est une relation d'équivalence.

Preuve Une relation d'équivalence est réflexive, symétrique et transitive :

- Réflexivité : oui, en ne faisant aucun renommage.
- Symétrie : en appliquant les renommages inverses.
- Transitivité : si $e_1 \equiv_{\alpha} e_2$, $e_2 \equiv_{\alpha} e_3$, on renomme e_1 en e_2 par les premiers renommages, puis e_2 en e_3 par les deuxièmes renommages, et donc $e_1 \equiv_{\alpha} e_3$

Théorème La relation d' α -équivalence est une relation dite « de congruence », c'est-à-dire, si $e_1 \equiv_{\alpha} e_2$, alors

- $\lambda x. e_1 \equiv_{\alpha} \lambda x. e_2$
- $\forall e \in \Lambda, e e_1 \equiv_{\alpha} e e_2$
- $\forall e \in \Lambda, e_1 e \equiv_{\alpha} e_2 e$

Preuve Pas besoin de détailler, il suffit juste de renommer la partie du λ -terme qui est importante.

1.3 Substitution

Une opération similaire à l' α renommage est celle de substitution, consiste à renommer des variables non pas en autre variables, mais en d'autres λ -termes. C'est exactement ce que l'on fait avec des formules logiques.

Si j'ai une suite (a_n) qui converge vers l , alors

$$\forall \epsilon > 0, \exists n_0 \in \mathbb{N} \mid \forall n \geq n_0, |a_n - l| < \epsilon$$

donc en prenant une limite $l = 0$ (et une suite qui converge bien vers 0) :

$$\forall \epsilon > 0, \exists n_0 \in \mathbb{N} \mid \forall n \geq n_0, |a_n - 0| < \epsilon$$

On peut dire qu'on a réécrit la formule avec la substitution $[l \leftarrow 0]$. Par contre, il faut faire attention : si j'avais appelé ma limite n_0 , j'aurais obtenu une formule qui n'a plus de sens, car n_0 est lié dans la formule.

Le principe est le même avec les λ -termes : on va remplacer des variables avec de nouveaux termes, à condition de ne pas transformer des variables liés en variables libres, ou vice-versa.

Définition Soit x une variable et $a, b \in \Lambda$ deux lambda-termes. On définit par induction la substitution par b de x dans le terme a , que l'on note

$$a[x \leftarrow b]$$

par

$$\begin{array}{ll} x[x \leftarrow b] = b & \text{on remplace } x \text{ par } b \\ y[x \leftarrow b] = y & \text{si } x \neq y, \text{ pour ne pas remplacer la mauvaise variable} \\ (a_1 a_2)[x \leftarrow b] = a_1[x \leftarrow b] a_2[x \leftarrow b] & \text{on remplace dans les deux termes de l'application} \\ (\lambda x. a_1)[x \leftarrow b] = \lambda x. a_1 & \text{on ne peut pas substituer une variable déjà liée} \\ (\lambda y. a_1)[x \leftarrow b] = \lambda y. (a_1[x \leftarrow b]) & \text{si } y = x \text{ et } y \notin \text{FV}(b), \text{ pour ne pas lier des } y \text{ venant de } b \end{array}$$

Lorsqu'une condition pour une substitution n'est pas remplie, on peut d'effectuer un α -renommage pour éviter d'avoir des variables en commun.

Remarque On peut lire la notation

$$a[x \leftarrow b]$$

« la substitution de x par b dans a », ou « a , en remplaçant toutes les occurrences libres de x par b », etc. Par ailleurs, on retrouve aussi les notations équivalentes dans la littérature :

$$a^{\{x \leftarrow b\}}$$

$$a[b/x]$$

Exemple

$$(x y)[y \leftarrow \lambda z. x] = x(\lambda z. x)$$

Exemple Dans cet exemple, on applique la substitution de manière incorrecte.

$$\underbrace{(\lambda x. x y)}_{\substack{x \text{ lié} \\ x \text{ libre}}} [y \leftarrow \lambda z. x] = \lambda x. \underbrace{(x [y \leftarrow \lambda z. x]) (y [y \leftarrow \lambda z. x])}_{\substack{\text{le } x \text{ qui était précédemment libre} \\ \text{est désormais un argument (lié)}}} \\ \text{???}$$

Pour pouvoir l'effectuer correctement, on réécrit le λ -terme d'origine en renommant la variable x liée (en t).

$$\lambda x. x y \equiv_{\alpha} \lambda t. t y$$

et donc

$$\begin{aligned} (\lambda t. t y)[y \leftarrow \lambda z. x] &= \lambda t. (t[y \leftarrow \lambda z. x]) (y[y \leftarrow \lambda z. x]) \\ &= \lambda t. t (\lambda z. x) \quad \text{ok} \end{aligned}$$

Très vite, on va se rendre compte que cette opération va nous permettre enfin de manipuler les lambda-termes en tant que fonctions. En effet, lorsque l'on applique une fonction f à une valeur y , on remplace les occurrences libres de x par y , et on enlève le « $f(x) =$ ». Donc, pour un λ -terme, pour effectuer une application, on enlève le « $\lambda x.$ », puis on remplace x par sa nouvelle valeur dans le reste.

1.4 Exécution par β -réduction

Dès que l'on a un lambda-terme qui est l'application d'un terme b à une lambda abstraction $\lambda x. a$, on peut "évaluer" ce terme en calculant l'application : on remplace x par b dans a (avec une substitution). Et ensuite, il n'y a plus qu'à répéter l'opération sur les nouveaux termes qui apparaissent. Ainsi, par exemple, le lambda-terme

$$(\lambda x. f x) y$$

deviendrait

$$f y$$

Définissons un opérateur qui met en relation deux λ -termes par ce procédé : c'est la β -réduction.

Définition D'abord, on appelle un β -rédex un λ -terme de la forme:

$$(\lambda x. a) b$$

...où $a, b \in \Lambda$, et x est une variable.

On note $\longrightarrow$ la relation de β -réduction, et on commence à mettre en relation tous les β -rédex avec un nouveau terme associé :

$$\underline{(\lambda x. a) b} \longrightarrow a [x \leftarrow b]$$

Puis pour chaque λ -terme contenant au moins un rédex, on le met en relation avec lui-même, où on a remplacé le rédex par son terme associé selon la définition ci-dessus.

$$\left(\lambda t. \dots \left(\underline{(\lambda x. a) b} \right) \dots \right) \longrightarrow \left(\lambda t. \dots (a [x \leftarrow b]) \dots \right)$$

Ceci constitue en son intégralité la relation de β -réduction. On notera également sa clôture réflexive et transitive $\longrightarrow^*$

Remarque Lorsque l'on effectue une étape de β -réduction, on peut souligner le rédex que l'on réduit pour le mettre en évidence.

Exemple

$$\begin{aligned} \text{id } y &= \underline{(\lambda x. x) y} \\ &\longrightarrow x [x \leftarrow y] \\ &= y \\ \text{id id} &= \underline{(\lambda x. x) (\lambda x. x)} \\ &\longrightarrow (\lambda x. x) = \text{id} \end{aligned}$$

Exemple

$$\begin{aligned} \underline{(\lambda x. \lambda y. \lambda z. y) a b c} &\longrightarrow \underline{(\lambda y. \lambda z. y) b c} \\ &\longrightarrow \underline{(\lambda z. b) c} \\ &\longrightarrow b \end{aligned}$$

On note dans toute la suite

$$\text{app} = \lambda f. \lambda x. f x$$

C'est le lambda-terme qui accepte un terme f , puis un argument x , et qui renvoie le terme $f x$. C'est donc la fonction qui prend une fonction f , et qui l'applique à une argument x voulu.

Exemple

$$\begin{aligned}\text{app id } a &= (\lambda f. \lambda x. f x) (\lambda t. t) a \\ &\longrightarrow (\lambda x. (\lambda t. t) x) a \\ &\longrightarrow (\lambda t. t) a \\ &\longrightarrow a\end{aligned}$$

On aurait pu le faire d'une autre manière :

$$\begin{aligned}\text{app id } a &= (\lambda f. \lambda x. f x) (\lambda t. t) a \\ &\longrightarrow (\lambda x. (\lambda t. t) x) a \\ &\longrightarrow (\lambda x. x) a \\ &\longrightarrow a\end{aligned}$$

Voici un exemple où l'on peut réduire une infinité de fois :

Exemple Notons $\Delta = \lambda x. x x$ et calculons le terme suivant :

$$\begin{aligned}\Delta \Delta &= (\lambda x. x x) (\lambda x. x x) \\ &\longrightarrow (x [x \leftarrow (\lambda x. x x)]) (x [x \leftarrow (\lambda x. x x)]) \\ &= (\lambda x. x x) (\lambda x. x x) \\ &= \Delta \Delta\end{aligned}$$

Donc $\Delta \Delta \longrightarrow^* \Delta \Delta$. On dit que ce λ -terme diverge.

Enfin, pour pouvoir exprimer que deux λ -termes sont « égaux » dans le sens où ils se comportent de la même manière, on peut définir une relation d'équivalence sur Λ avec la β -réduction.

Définition On désigne par $\equiv_\beta$ la clôture réflexive, transitive et symétrique de la β -réduction. C'est-à-dire que c'est la plus petite relation telle que :

- $\forall (e, e') \in \Lambda^2, e \longrightarrow e' \implies e \equiv_\beta e'$
- **Réflexivité** : $\forall e \in \Lambda, e \equiv_\beta e$
- **Symétrie** : $\forall (e, e') \in \Lambda^2, e \equiv_\beta e' \iff e' \equiv_\beta e$
- **Transitivité** : $\forall (a, b, c) \in \Lambda^3, a \equiv_\beta b \text{ et } b \equiv_\beta c \implies a \equiv_\beta c$

Autrement dit, intuitivement, deux lambda-termes sont β -équivalents (« égaux ») ils peuvent s'évaluer à un même lambda-terme. C'est un peu la même chose que pour les calculs mathématiques : deux expressions mathématiques sont égales si je peux appliquer des règles de calcul des deux côtés pour arriver au même résultat :

$$\underbrace{(1 + 1) = 2}_{\text{par addition sur les entiers}} \quad \text{et} \quad \underbrace{(3 - 1) = 2}_{\text{par soustraction sur les entiers}} \quad \text{donc} \quad \ll 1 + 1 = 3 - 1 \gg$$

Théorème Conséquence : si $a \longrightarrow^* c$ et $b \longrightarrow^* c$, alors $a \equiv_\beta c$.

Preuve $a = a_1 \longrightarrow a_2 \longrightarrow \dots \longrightarrow c$, donc $a = a_1 \equiv_\beta a_2 \equiv_\beta \dots \equiv_\beta c$, d'où $a \equiv_\beta c$ par transitivité, et pareil pour $b \equiv_\beta c$. Encore par transitivité, on a $a \equiv_\beta b$.

La relation $\equiv_\beta$ est clairement une relation d'équivalence étant donné sa définition.

1.5 Stratégie de réduction

Les quelques derniers exemples montrent qu'il y a plusieurs façons d'évaluer le même λ -terme par β -réduction, et que, parfois, on peut ne jamais s'arrêter. On peut alors mettre en place plusieurs stratégies qui dictent les redex qu'on a le droit de réduire ou pas.

Définition Un λ -terme est dit sous forme *normale*, si, pour une stratégie de réduction donnée, il n'existe aucun autre terme obtenu par β -réduction.

La stratégie dite « *appel par nom* », aussi notée *CALL-BY-NAME*, impose que l'on évalue toujours les redex les plus à gauche, les plus extérieurs. Cette stratégie a pour conséquence que l'on évalue substitue toujours un argument par son terme associé lorsque l'on évalue une fonction. Cette stratégie a le mérite de toujours atteindre sa forme normale lorsqu'elle existe.

Une autre stratégie, appelée « *appel par valeur* », notée *CALL-BY-VALUE*, impose, elle, que l'on réduise les arguments d'une fonction avant de les appliquer. Cette stratégie est celle employée par la plupart des langages de programmation, y compris C et OCaml.

Exemple Le λ -terme suivant est sous forme normale selon la stratégie *CALL-BY-VALUE*, car on ne peut pas évaluer l'intérieur de la fonction sans d'abord l'appliquer à tous les arguments.

$$(\lambda y. (\lambda x. f\ x)\ y)$$

Même s'il y a un redex, $(\lambda x. f\ x)\ y$, dans la λ -abstraction, la stratégie interdit sa réduction, et donc il n'y a rien à faire.

2 Programmer avec le lambda-calcul

On a vu comment utiliser le λ -calcul comme outil permettant de modéliser le calcul d'applications de fonctions à d'autres fonctions et d'autres variables. Mais si les fonctions sont les seules primitives du langage, est-il vraiment utile ? Que peut-on faire d'autre comme calcul qui ressemble plus à ceux d'un vrai langage de programmation ?

2.1 Booléens de Church

Dans le λ -calcul, on n'a que les fonctions et les variables comme valeurs, mais en OCaml, les booléens sont deux constantes, *true* et *false*, qui constituent le type *bool*. Pour pouvoir faire l'analogie, il faudrait définir un λ -terme qui correspond à *true*, et un autre qui correspond à *false*.

Nous allons voir deux lambda-termes qui remplissent parfaitement ce devoir. Ils ont été utilisés pour la première fois par Alonzo Church.

Définition

$$\begin{aligned}\text{true} &= \lambda x. \lambda y. x \\ \text{false} &= \lambda x. \lambda y. y\end{aligned}$$

Ainsi, le booléen *true* est représenté par la fonction qui prend deux arguments, *x* et *y*, et qui renvoie juste le premier, *x*. Similairement, le booléen *false* est représenté par la fonction qui prend deux arguments *x* et *y*, mais qui ne renvoie que le deuxième, *y*.

Pour comprendre cette représentation, considérons le fonctionnement des booléens en OCaml. Que peut-on faire avec des booléens ? Ce sont des valeurs simples, que l'on peut stocker et manipuler, ou même combiner avec des portes logiques ou autres formules. Mais principalement, les booléens servent à effectuer des tests dans un programme. Comme dans, par exemple :

```
if condition then x else y
```


Lorsque cette expression est évaluée, il y a deux cas possibles :

- `condition = true` : alors on a `if true then x else y`, qui donne simplement `x`.
- `condition = false` : similairement, on a `if false then x else y`, soit juste `y`.

En quelques sortes, la structure de contrôle `if-then-else` en OCaml agit comme si l'on avait appliqué `x` et `y` à une fonction qui choisit soit `x` soit `y` : et cette fonction n'est autre que `true`, ou `false` telles que nous les avons définies. Si bien que l'on peut redéfinir cette structure de contrôle :

$$\text{if} = \lambda b. \lambda x. \lambda y. b \ x \ y$$

Ici, ce λ -terme prend un lambda-terme b en argument, et l'applique à deux autres arguments x et y . Justement, si b se trouve être `true`, alors on a

$$\begin{aligned} \text{if true } a \ b &= \underline{\underline{(\lambda b. \lambda x. \lambda y. b \ x \ y) \ \text{true } a \ b}} \\ &\longrightarrow^* \text{true } a \ b \\ &= \underline{(\lambda x. \lambda y. x) \ a \ b} \\ &\longrightarrow \underline{(\lambda y. a) \ b} \\ &\longrightarrow a \end{aligned}$$

Ce qui est bien le résultat que l'on cherchait à obtenir ! Similairement :

$$\begin{aligned} \text{if false } t \ f &\longrightarrow^* \text{false } t \ f \\ &= (\lambda x. \lambda y. y) \ t \ f \\ &\longrightarrow^* f \end{aligned}$$

On peut alors commencer à construire des lambda-termes qui commencent à ressembler à des expressions d'un langage de programmation, et même définir des opérations que l'on connaît déjà :

Définition Opération sur les booléens :

$$\begin{aligned} \text{not} &= \lambda b. \text{if } b \ \text{false} \ \text{true} \\ \text{and} &= \lambda a. \lambda b. \text{if } a \ b \ \text{false} \\ \text{or} &= \lambda a. \lambda b. \text{if } a \ \text{true} \ b \\ \text{xor} &= \lambda a. \lambda b. \text{or } (\text{and } a \ (\text{not } b)) \ (\text{and } b \ (\text{not } a)) \end{aligned}$$

Exemple

$$\begin{aligned} \text{if } (\text{and } \text{true} \ \text{false}) \ x \ (\text{if } (\text{or } \text{false} \ \text{true}) \ y \ z) &\longrightarrow^* \text{if false } x \ (\text{if } (\text{or } \text{false} \ \text{true}) \ y \ z) \\ &\longrightarrow^* \text{if } (\text{or } \text{false} \ \text{true}) \ y \ z \\ &\longrightarrow^* \text{if true } y \ z \\ &\longrightarrow^* \text{true } y \ z \\ &\longrightarrow^* y \end{aligned}$$

Remarque On peut remarquer qu'à chaque fois que l'on écrit « `if` », il est redondant puisqu'il disparaît éventuellement. En effet, et la raison pour cela est que

$$\text{if } b \ x \ y \equiv_{\beta} b \ x \ y$$

car

$$\begin{aligned} (\lambda b. \lambda x. \lambda y. b \ x \ y) \ b' \ x' \ y' &\longrightarrow (\lambda x. \lambda y. b' \ x \ y) \ x' \ y' \longrightarrow (\lambda y. b' \ x' \ y) \ y' \\ &\longrightarrow b' \ x' \ y' \end{aligned}$$

2.2 Entiers naturels de Church

Comme avec les booléens, on a vu que l'on pouvait se servir de lambda-termes pour représenter les valeurs « vrai » ou « faux ». Observons que pour cela, on a utilisé les variables liées dans une λ abstraction pour « stocker des informations » :

$$\text{true} = \underbrace{\lambda x. \lambda y.}_{\text{valeurs stockées dans ces variables}} x$$

Plus précisément, un « booléen » de Church stocke la valeur à renvoyer s'il est vrai (dans x), et stocke la valeur à renvoyer s'il est faux (dans y).

Ainsi, si on arrive à utiliser dans lambda-termes que l'on stocke dans une abstraction, qui permettent de calculer un entier, alors on pourra utiliser des lambda-termes comme des entiers ! Mais d'abord, qu'est-ce qu'un entier ?

Définition Définissons les entiers naturels $\mathbb{N}$ de la manière suivante comme un ensemble inductif :

$$n := 0 \mid S(n)$$

où 0 est un constructeur d'arité 0, et S est un constructeur d'arité 1.

Donc selon cette définition, on a $3 = S(S(S(0)))$, ou bien $N = \underbrace{S(S(\dots(S(0))))}_{N \text{ fois}}$

On peut alors le voir sous un autre angle. Pour construire un entier N , on se donne une fonction « successeur » S , et on l'applique N fois à un élément initial « 0 », très exactement comme une suite récurrente définie par une fonction, de terme initial 0. Ceci est avantageux car le λ -calcul modélise parfaitement le calcul d'applications de fonctions. On va alors créer un λ -terme qui, étant donné deux arguments S et z (zéro), appliquera S un certain nombre de fois à z pour *représenter* un entier naturel :

Commençons simplement, pour calculer zéro, j'applique zéro fois la fonction successeur à zéro lui-même.

$$0 = \lambda S. \lambda z. z$$

Notons que la définition de 0 comme λ -terme est bien une fonction, et pas juste z : sinon, je ne pourrais rien faire avec z , ni S ! (ce sont juste des variables). Je poursuis avec l'entier suivant, il faut appliquer une fois la fonction successeur à zéro :

$$1 = \lambda S. \lambda z. S z$$

Puis deux, trois, ..., N fois :

Définition

$$0 = \lambda S. \lambda z. z$$

$$1 = \lambda S. \lambda z. S z$$

$$2 = \lambda S. \lambda z. S (S z)$$

$$3 = \lambda S. \lambda z. S (S (S z))$$

$\vdots$

$$\boxed{N} = \lambda S. \lambda z. \underbrace{S (S (\dots S (S z)))}_{N \text{ fois}}$$

où $\boxed{N}$ désigne l'écriture décimale de N et non le caractère littéral « N »

Ces λ -termes sont la représentation des entiers naturels de Church, aussi appelés les « entiers de Church ».

Remarquons tout de suite quelque chose : tous mes entiers sont toujours de la forme $\lambda S. \lambda z. (\dots)$ où S est appliqué un nombre fini de fois à z dans les parenthèses. Ceci est important, et nous aidera à savoir si nous sommes en train d'écrire un entier ou non. Si nous voulons créer un entier de Church, il faudra s'assurer que le lambda terme que l'on écrit est bien de cette même forme.

Tentons ainsi de définir les opérations usuelles sur les entiers naturels avec cette nouvelle définition.

2.2.1 Successeur

Que faut-il faire pour incrémenter un entier ? Par définition, il faut appliquer la fonction successeur S . Donc, si j'ai un lambda-terme $n = \lambda S. \lambda z. (\dots)$, qui applique n fois S à z , il faut que j'applique une dernière fois S au résultat de l'appel $n S z$.

La fonction successeur sur les entiers de Church ressemble donc à ceci:

$$\text{succ} = \lambda n. \lambda S. \lambda z. S (n S z)$$

Il peut parfois être difficile de comprendre ce que fait un lambda-terme juste en regardant son écriture. Détaillons plus spécialement la β -réduction d'un appel à succ .

- Le premier argument, $(\lambda n. (\dots))$ est l'entier de Church dont on prend le successeur : il faut donc se rappeler que c'est un lambda-terme de la forme

$$\lambda S. \lambda z. (\dots)$$

qui calcule $S^n(z)$.

- Il n'y a aucun autre argument à prendre, donc on doit maintenant retourner un entier de Church.

$$\lambda n. \underbrace{\dots}_{\text{entier de Church}}$$

Ainsi, le lambda-terme qui suit l'argument n doit être de la forme $\lambda S. \lambda z. (\dots)$, qui applique $n + 1$ fois S à la constante z . On commence donc par écrire les arguments λS et λz :

$$\underbrace{\lambda n.}_{\text{argument } n} \left(\underbrace{\lambda S. \lambda z.}_{\text{écriture d'un entier de Church}} \left(\underbrace{(\dots)}_{\text{calcul de } S^{n+1}(z)} \right) \right)$$

- Pour le calcul de $S^{n+1}(z)$, on peut s'appuyer sur le calcul qu'effectue n lorsqu'on lui passe les arguments S et z . Par définition, $n S z$ calcule $\underbrace{S(S(\dots S(z)))}_{n \text{ fois}}$. Il ne reste plus qu'à appliquer S une dernière fois !

$$S (n S z)$$

...ce qui donne bien $\underbrace{S(S(S(\dots S(z))))}_{n \text{ fois}}_{n+1 \text{ fois}}$, c'est à dire, par définition, $\boxed{n+1}$!

Pour résumer :

$$\text{succ} = \underbrace{\lambda n.}_{\text{argument } n} \left(\underbrace{\lambda S. \lambda z.}_{\text{écriture d'un entier de Church}} \left(\underbrace{S \left(\underbrace{n S z}_{\text{calcul de } S^n(z)} \right)}_{\text{calcul de } S^{n+1}(z)} \right) \right)$$

Exemple Pour comprendre la réduction d'un appel à succ , regardons en détail cet exemple

$$\begin{aligned} \text{succ } 3 &= (\lambda n. \lambda S. \lambda z. S (n S z)) (\lambda S. \lambda z. S(S(S z))) \\ &\longrightarrow \lambda S. \lambda z. S \left((\lambda S. \lambda z. S(S(S z))) S z \right) \\ &\longrightarrow^* \lambda S. \lambda z. S (S(S(S z))) \quad \text{qui est bien sous la forme} \\ &\equiv_{\beta} 4 \quad \text{d'un entier de Church !} \end{aligned}$$

2.2.2 Addition

Similairement, à la place de juste calculer le successeur une dernière fois sur un entier de Church, on peut utiliser deux entiers de Church n et m pour calculer les $n + m$ appels à S sur zéro. En effet, pour visualiser cela, on peut utiliser une propriété usuelle sur les puissances de fonctions :

$$f^{a+b}(x) = f^a f^b(x) = f^a(f^b(x))$$

Donc, pour calculer l'entier $n + m$ à l'aide des successeurs :

$$\begin{cases} n = S^n(0) \\ m = S^m(0) \end{cases} \implies n + m = S^n(S^m(z))$$

D'après ce qui a été vu avec le λ -terme successeur, il est facile de construire le λ -terme qui performe l'addition de deux entiers de Church :

$$\text{add} = \lambda n. \lambda m. \lambda S. \lambda z. n S (m S z)$$

Encore une fois, il est compliqué de comprendre ce que fait un terme juste en regardant son écriture, donc voici une écriture plus détaillée (je ne vais pas repasser sur la même explication pour la fonction successeur) :

$$\text{add} = \underbrace{\lambda n. \lambda m.}_{\text{argument } n \text{ et } m \text{ de l'addition}} \left(\underbrace{\lambda S. \lambda z.}_{\text{écriture d'un entier de Church}} \left(\underbrace{n S \left(\underbrace{m S z}_{\text{calcul de } S^m(z)} \right)}_{\text{calcul de } S^{n+m}(z)} \right) \right)$$

Une autre façon de le voir est que, dans le calcul $n S (m S z)$, le terme $m S z$ devient le « nouveau zéro » pour le calcul de S^n (nouveau zéro), sauf que ce zéro est en fait l'entier m . Cela a pour effet d'appliquer n fois S à ce nouveau zéro, et par extension, $n + m$ fois S au vrai zéro (0).

Exemple Pour le voir avec un exemple (addition de 2 et 3):

$$\begin{aligned} \text{add } 2 \ 3 &= \underbrace{(\lambda n. \lambda m. \lambda S. \lambda z. n S (m S z))}_{\text{add}} \underbrace{(\lambda S. \lambda z. S(S z))}_2 3 \\ &\longrightarrow^* \lambda S. \lambda z. \underbrace{(\lambda S. \lambda z. S(S z))}_2 S (3 S z) \\ &\longrightarrow^* \lambda S. \lambda z. \underbrace{(\lambda S. \lambda z. S(S z))}_2 S (3 S z) \\ &\longrightarrow^* \lambda S. \lambda z. S(S(3 S z)) \\ &= \lambda S. \lambda z. S(\underbrace{S((\lambda S. \lambda z. S(S(S z))))}_3 S z) \\ &\longrightarrow^* \lambda S. \lambda z. S(S(S(S(S z)))) \\ &\equiv_\beta 5 \end{aligned}$$

Donc « $2 + 3 = 5$ », ...enfin, $\text{add } 2 \ 3 \equiv_\beta 5$

Le λ -terme d'addition va nous permettre d'exprimer les autres opérations sur les entiers. Même la fonction successeur : en effet, le successeur d'un entier n est $n + 1$.

Théorème Expression de succ à l'aide de add :

$$\text{succ} \equiv_{\beta} \text{add } 1$$

Autrement dit, la fonction successeur est parfaitement (β -)équivalente à la fonction qui ajoute 1.

Preuve On vérifie par calcul que add 1 est bien β -équivalent à succ :

$$\begin{aligned} \text{add } 1 &= (\lambda n. \lambda m. \lambda S. \lambda z. n \ S \ (m \ S \ z)) \ (\lambda S. \lambda z. S \ z) \\ &\longrightarrow \lambda m. \lambda S. \lambda z. \underline{(\lambda S. \lambda z. S \ z) \ S \ (m \ S \ z)} \\ &\longrightarrow^* \lambda m. \lambda S. \lambda z. S(m \ S \ z) \\ &\equiv_{\alpha} \lambda n. \lambda S. \lambda z. S(n \ S \ z) \\ &= \text{succ} \end{aligned}$$

D'où le résultat.

2.2.3 Multiplication

De la même manière, on utilise les propriétés de calcul sur les puissances des fonctions pour se donner une idée de comment écrire un lambda-terme de multiplication sur les entiers naturels de Church.

$$f^{ab}(x) = (f^a)^b(x) = \underbrace{f^a(f^a(\dots f^a(x)))}_{b \text{ fois}}$$

On remarque que cette écriture ressemble beaucoup à $S^m(z)$, sauf que le « nouveau successeur » devient S^n , qui n'est autre que la fonction qui ajoute n à un entier. Cela paraît évident : la multiplication est en réalité la répétition d'une addition. En utilisant donc add défini précédemment, on peut définir mul :

$$\text{mul} = \lambda n. \lambda m. m \ (\text{add } n) \ 0$$

Comme toujours, voici une écriture détaillée, qui bizarrement, cette fois, a l'air plus simple :

$$\text{mul} = \underbrace{\lambda n. \lambda m.}_{\substack{\text{arguments } n \text{ et } m \\ \text{de la multiplication}}} \left(\underbrace{\begin{matrix} m & (\text{add } n) & 0 \\ & \text{nouveau successeur } S^n \\ & \text{qui fait } +n \end{matrix}}_{\text{calcul de } S^{m \times n}(z)} \right)$$

Quelque chose a changé par rapport aux autres opérateurs usuels... il n'y a plus l'écriture d'un entier de Church ($\lambda S. \lambda z.$) après les arguments n et m . De même, à cause de cela, le zéro z n'est plus dans l'écriture, et a été remplacé par le λ -terme 0. Pourquoi ?

Pour comprendre cela, rappelons nous qu'un entier de Church est en réalité une fonction qui à f et x associe $f^n(x)$ (où n est un certain entier naturel). Sous l'écriture mathématiques :

$$\boxed{n} = (f, x) \mapsto \underbrace{f(f(\dots f(x)))}_{n \text{ fois}}$$

Donc lorsque l'on écrit $m \ (\text{add } n) \ 0$, on calcule l'application de add n , m fois à zéro. Or, le λ -terme add n prend un argument un autre λ -terme sous la forme d'un entier de Church. Ici, on commence à partir de zéro, donc il faut l'appliquer à l'entier de Church nul 0, et non la constante z !

Exemple

$$\begin{aligned}
 \text{mul } 4 \ 5 &\longrightarrow^* 4 \ (\text{add } 5) \ 0 \\
 &= (\lambda S. \lambda z. S(S(S(S \ z)))) \ (\text{add } 5) \ 0 \\
 &\longrightarrow^* (\text{add } 5)((\text{add } 5)((\text{add } 5)(\text{add } 5 \ 0))) \\
 &\equiv_\beta (\text{add } 5)((\text{add } 5)(\text{add } 5 \ 5)) \\
 &\equiv_\beta (\text{add } 5)(\text{add } 5 \ 10) \\
 &\equiv_\beta \text{add } 5 \ 15 \\
 &\equiv_\beta 20
 \end{aligned}$$

Donc $\text{mul } 4 \ 5 \equiv_\beta 20$, c'est-à-dire $4 \times 5 = 20$.

2.2.4 Exponentiation

Avec la construction du dernier opérateur, on peut généraliser la construction du lambda-terme pour l'opération qui répète une autre opération précédente déjà définie. En l'occurrence, l'exponentiation a^b est la répétition b fois de la multiplication par a , en partant du neutre 1.

$$\text{exp} = \lambda n. \lambda m. m \ (\text{mul } n) \ 1$$

Exemple

$$\begin{aligned}
 \text{exp } 2 \ 3 &\longrightarrow^* (\text{mul } 2)((\text{mul } 2)(\text{mul } 2 \ 1)) \\
 &\equiv_\beta (\text{mul } 2)(\text{mul } 2 \ 2) \\
 &\equiv_\beta \text{mul } 2 \ 4 \\
 &\equiv_\beta 8
 \end{aligned}$$

Donc $\text{exp } 2 \ 3 \equiv_\beta 8$, c'est-à-dire $2^3 = 8$.

Remarque On peut généraliser ce procédé pour n'importe quelle opération qui répète une opération précédemment définie.

Par exemple, la *tétration* est la répétition de l'exponentiation, et se note ${}^b a$, ou encore $a \uparrow\uparrow b$. Donc, pour $2 \uparrow\uparrow 4$, on a $2^{2^{2^2}} = 2^{2^4} = 2^{16} = 65\,536$. Et le lambda-terme qui correspond à la tétration est :

$$\text{tetra} = \lambda n. \lambda m. m \ (\text{exp } n) \ 1$$

Puis ensuite, il y a la pentation, qui se note $a \uparrow\uparrow\uparrow b$.

Exemple : $2 \uparrow\uparrow\uparrow 4 = 2 \uparrow\uparrow 2 \uparrow\uparrow 2 \uparrow\uparrow 2 = 2 \uparrow\uparrow 2 \uparrow\uparrow 4 = 2 \uparrow\uparrow 65\,536 = \underbrace{2^{2^{2^{2^{\dots}}}}}_{65536 \text{ fois}}$ qui est astronomiquement grand. Ce qui se calcule par :

$$\text{penta} = \lambda n. \lambda m. m \ (\text{tetra } n) \ 1$$

Enfin, on peut noter $\uparrow^p$ l'opérateur obtenu répétant cette construction p fois. (Exemple : $\uparrow\uparrow\uparrow = \uparrow^3$)

$$\forall p \in \mathbb{N}, \uparrow^{p+1} = \lambda n. \lambda m. m \ (\uparrow^p n) \ 1$$

2.3 Listes finies

L'étude des entiers de Church nous a montré que les fonction itérées pouvait être bien utile pour effectuer des calculs. En particulier, à l'aide d'une fonction S itérée un certain nombre de fois sur une constante z , on a pu construire tous les entiers naturels, ainsi qu'une infinité d'opérations sur ces derniers.

Avec ce nouveau pouvoir, on va construire quelque chose de plus général: les listes finies. Mais pour cela, il faut d'abord définir leur structure.

Définition Soit A un ensemble. On définit l'ensemble des listes finies, noté $\text{List}(A)$ comme un ensemble inductif :

$$L := \text{Empty} \quad | \quad \text{Cons}(x, L) \quad \text{où } x \in A$$

Empty est un constructeur constant (d'arité 0), et Cons est un constructeur d'arité 2, de la forme $A \times L \rightarrow L$.

Remarque Cette définition correspond à la définition des listes en OCaml.

$$\text{type 'a list} = \underbrace{[]}_{\text{Empty}} \quad | \quad \underbrace{(::)}_{\text{Cons}} \text{ of 'a * 'a list}$$

Exemple La liste $[1, 2, 3]$ est représentée par $\text{Cons}(1, \text{Cons}(2, \text{Cons}(3, \text{Empty})))$.

Exactement comme pour les entiers naturels, on va se munir de deux lambda-termes correspondant à Empty et Cons , puis on va les utiliser pour calculer des listes à l'aide d'applications sur ces lambda-termes.

On appellera lambda-termes C et e . Ainsi, les listes de $\text{List}(A)$ seront des lambda-termes de la forme suivante

$$\lambda C. \lambda e. C a_1 (C a_2 \dots (C a_n e)) \quad \text{où } a_1, \dots, a_n \text{ sont les } n \text{ éléments de la liste}$$

Encore une fois, C et e sont des variables arbitraires qui sont des arguments dans des λ -abstractions, et non pas des constantes, car sinon on ne pourrait rien faire avec ! Il se s'agit pas de définir ce que font C et e , mais plutôt de les utiliser pour décrire la forme d'une liste.

Dans toute la suite, je m'autorise à utiliser la notation $[a_1, \dots, a_n]$ pour désigner des listes représentées par un lambda-terme.

Définition

$$\text{empty} = \lambda C. \lambda e. e$$

qui correspond à la liste $[]$

$$\text{cons} = \lambda x. \lambda L. \lambda C. \lambda e. C x (L C e)$$

qui construit la liste $\text{Cons}(x, L)$
à partir de $x \in A$ et d'une liste L

Le lambda-terme empty est simple à comprendre. C'est la représentation de la liste vide $[]$: étant donné le constructeur C et le symbole vide e , il renvoie le symbole vide. On remarquera que ce lambda-terme est α -équivalent à 0 et à false : cela n'a rien à voir, mais il se trouve que le fonctionnement de ces objets est le même.

Cependant, comprendre cons est moins évident. Une écriture détaillée permet de mieux le voir :

$$\text{cons} = \underbrace{\lambda x. \lambda L.}_{\substack{\text{l'élément } x \in A \text{ à ajouter en tête} \\ \text{et la liste } L \text{ qui deviendra la queue}}} \left(\underbrace{\lambda C. \lambda e.}_{\text{écriture d'une liste}} \left(\underbrace{C x \quad (L C e)}_{\substack{\text{calcul de la liste } L \\ \text{dernière construction avec } x \text{ en tête}}} \right) \right)$$

Ceci devrait paraître similaire à la construction des entiers de Church. Les listes commencent par $(\lambda C. \lambda e.)$, toujours.

Exemple La liste $[1, 2, 3]$ (où 1, 2 et 3 sont des entiers de Church) est représentée par

$$\text{cons } 1 (\text{cons } 2 (\text{cons } 3 \text{ empty}))$$

qui est β -équivalente à

$$\lambda C. \lambda e. C 1 (C 2 (C 3 e))$$

2.3.1 Concaténation

En une liste L_2 comme une liste « vide » pour une autre liste L_1 , on peut effectuer la concaténation $L_1 L_2$ en calculant la liste L_2 , puis en la passant comme constructeur vide pour le calcul de L_1 .

$$\text{concat} = \lambda L_1. \lambda L_2. \lambda C. \lambda e. L_1 C (L_2 C e)$$

L'explication est la même que pour les entiers.

Exemple

$$\begin{aligned} \text{concat } [1,2,3] [4,5,6] &= (\lambda L_1. \lambda L_2. \lambda C. \lambda e. L_1 C (L_2 C e)) [1,2,3] [4,5,6] \\ &\longrightarrow^* \lambda C. \lambda e. [1,2,3] C ([4,5,6] C e) \\ &= \lambda C. \lambda e. [1,2,3] C ((\lambda C. \lambda e. C 4 (C 5 (C 6 e))) C e) \\ &\longrightarrow^* \lambda C. \lambda e. [1,2,3] C (C 4 (C 5 (C 6 e))) \\ &= \lambda C. \lambda e. (\lambda C. \lambda e. C 1 (C 2 (C 3 e))) C (C 4 (C 5 (C 6 e))) \\ &\longrightarrow^* \lambda C. \lambda e. C 1 (C 2 (C 3 (C 4 (C 5 (C 6 e)))))) \\ &= [1,2,3,4,5,6] \end{aligned}$$

Donc $\text{concat } [1,2,3] [4,5,6] \equiv_\beta [1,2,3,4,5,6]$ comme attendu.

2.3.2 Aggrégation / réduction / fold

La fonction Ocaml `fold_right` prend un argument une fonction $f : 'a \rightarrow 'acc \rightarrow 'acc$, un élément initial `init : 'acc`, et une liste de type `'a list` de la forme $[a_1, \dots, a_n]$ et renvoie :

$$f(a_1, f(\dots, f(a_n, \text{init})))$$

ce qui correspond exactement à

$$C a_1 (\dots (C a_n e))$$

où on a remplacé f par C , et `init` par e . Ainsi, si l'on veut effectuer la même opération que `fold_right` avec une liste sous la forme d'un λ -terme, on peut simplement utiliser la fonction :

$$\text{fold}_R = \lambda f. \lambda i. \lambda L. L f i$$

Ceci aura l'effet de remplacer tous les C par f et le e final par i , et calculera donc l'aggrégation par la droite (fold-right) de la liste.

Exemple Avec des entiers naturels de Church :

$$\begin{aligned} \underline{\underline{\text{fold}_R \text{ add } 0 [1,2,3,4,5]}} &\longrightarrow^* [1,2,3,4,5] \text{ add } 0 \\ &= (\lambda C. \lambda e. C 1 (C 2 (C 3 (C 4 (C 5 e)))) \text{ add } 0) \\ &\longrightarrow^* \text{add } 1 (\text{add } 2 (\text{add } 3 (\text{add } 4 (\text{add } 5 0)))) \\ &\longrightarrow^* \dots \\ &\longrightarrow^* 15 \end{aligned}$$

Exemple Avec des booléens de Church, on définit des termes qui vérifient si tous les booléens d'une liste sont true, respectivement qu'il existe un moins un booléen true :

$$\begin{aligned} \text{all} &= \text{fold}_R \text{ and } \text{true} & \text{all } [\text{true}, \text{false}, \text{false}] &\equiv_\beta \text{false} \\ \text{any} &= \text{fold}_R \text{ or } \text{false} & \text{any } [\text{true}, \text{false}, \text{false}] &\equiv_\beta \text{true} \end{aligned}$$

Exemple Avec une liste de listes :

$$\text{flatten} = \text{fold}_R \text{ concat empty}$$

Ce λ -terme accepte une liste de listes en arguments et calcule la concaténation de toutes les listes de droite à gauche.

$$\begin{aligned} \text{flatten } [[1, 2], [0, 5, 7, 6], [], [9, 16]] &\equiv_{\beta} \text{concat } [1, 2] (\text{concat } [0, 5, 7, 6] (\text{concat } [] (\text{concat } [9, 16] []))) \\ &\equiv_{\beta} [1, 2, 0, 5, 7, 6, 9, 16] \end{aligned}$$

2.3.3 Map

On définit similairement une fonction qui à $(f, [a_1, \dots, a_n])$ associe $[f(a_1), \dots, f(a_n)]$.

$$\text{map} = \lambda f. \lambda L. \lambda C. \lambda e. L (\lambda x. \lambda l. C (f x) l) e$$

Le terme auxiliaire $\lambda x. \lambda l. C (f x) l$ remplace tous les C dans la liste L , ce qui a pour effet de remplacer tous les $C a_1 (X a_2 (\dots (C a_n e)))$ en $C (f a_1) (X (f a_2) (\dots (C (f a_n) e)))$

2.4 Tuples

Passons à quelque chose de plus différent : on va représenter les tuples à $N \geq 1$ éléments !

Comment peut-on calculer un tuple ? La méthode diffère, car on ne construit pas un tuple avec une fonction itérée, mais on les stocke plutôt en mémoire avant de choisir lesquels des éléments du tuple on veut obtenir. Commençons alors par écrire les λ -termes qui correspondent à choisir la k -ième valeur parmi n .

Définition

$$\forall k \in \llbracket 1, N \rrbracket, \quad \text{take}_{N,k} = \lambda x_1. \lambda x_2. \dots \lambda x_k. \dots \lambda x_N. x_k$$

C'est le lambda-terme qui accepte $x_1, x_2, \dots, x_k, \dots, x_N$ en arguments, et qui retourne simplement x_k .

On peut alors modéliser un tuple par un appel à une telle fonction : si j'ai des éléments $x_1, \dots, x_N$, alors je peux appliquer une fonction $\text{take}_{N,k}$ à ces N éléments pour choisir le k -ième. Enfin, il suffit de faire abstraction de cette fameuse fonction sélectrice pour représenter n'importe quel tuple.

On impose ainsi qu'un tuple de taille N soit toujours de la forme suivante :

$$\lambda s. s x_1 x_2 \dots x_N$$

Autrement dit, un tuple est un λ -terme qui accepte une fonction sélectrice et qui l'applique à N éléments (qui sont ses valeurs intérieures). Pour construire un tel tuple, on peut définir une fonction qui prend les éléments du tuples, et qui construit le tuple en question sous la forme imposée :

Définition

$$\text{tuple}_N = \lambda x_1. \dots \lambda x_N. (\lambda s. s x_1 \dots x_N)$$

$$\forall k \in \llbracket 1, N \rrbracket, \quad \text{select}_{N,k} = \lambda t. t \text{ take}_{N,k}$$

Le terme tuple_N nous permet de construire un tuple de manière opaque avec un simple appel à une fonction, et les fonctions select_k nous permettent de prendre un tel tuple argument, et d'en extraire le k -ième élément.

Je me permets d'utiliser la notation $(x_1, \dots, x_N)$ comme λ -terme.

Remarque Pour $N = 2$, on pourra noter

$$\text{select}_{2,1} = \text{fst}$$

$$\text{select}_{2,2} = \text{snd}$$

Exemple Quelques exemples simples.

Représentation d'un point de l'espace : $P = \text{tuple}_2 \ 5 \ 0 \ 1$, et ses coordonnées :

$$\text{select}_{3,1} P \equiv_{\beta} 5 \quad \text{select}_{3,2} P \equiv_{\beta} 0 \quad \text{select}_{3,3} P \equiv_{\beta} 1$$

Matrice de taille 2×3 : $M = \text{tuple}_2 \ (\text{tuple}_3 \ 9 \ 0 \ 1) \ (\text{tuple}_3 \ 8 \ 3 \ 6)$ qui vaut $\begin{pmatrix} 9 & 0 & 1 \\ 8 & 3 & 6 \end{pmatrix}$

Coefficient $(2, 3)$: $\text{select}_3 (\text{select}_2 M) \equiv_{\beta} 6$

2.4.1 Extension des nombres de Church

Les couples (tuples de taille 2) nous permettent de débloquent le pouvoir de la décrémentation. En effet, on peut maintenant représenter un entier non pas par le calcul de $S^n(z)$, mais plutôt par le couple $(S^n(z), S^{n-1}(z))$. Ainsi, le premier élément du couple représente l'entier en question, et le deuxième élément est son prédécesseur.

Pour éviter de redéfinir les mêmes opérateurs (succ, add, mul, exp, etc...) sur cette structure, on ne va tout de même pas changer la structure des entiers de Church. Plutôt, on va écrire des fonctions qui, lorsqu'elles ont besoin du prédécesseur d'un entier, calculent l'entier sous la nouvelle représentation (avec un couple), récupèrent le second élément, puis l'utilisent comme un entier de Church usuel.

Donnons-nous les deux λ -termes suivants :

- $z' = \text{tuple}_2 \ 0 \ 0$.

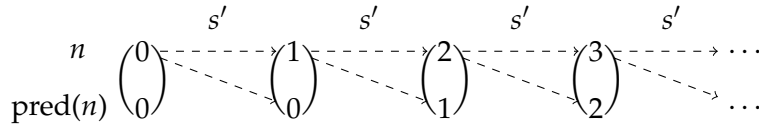
Ce λ -terme est un couple constitué de zéro et zéro. C'est le terme initial constant auquel on va appliquer notre nouvelle fonction successeur spécialement définie pour les entiers sous cette forme.

Pourquoi $(0, 0)$, alors que le prédécesseur de 0 est -1 ? Nous n'avons pas défini les entiers relatifs avec les λ -termes, donc on impose que le prédécesseur de 0 soit 0.

- $s' = \lambda n. \text{tuple}_2 \ (\text{succ} \ (\text{fst } n)) \ (\text{fst } n)$

Cette fonction prend en argument un couple d'entiers de Church $(_, n)$, et construit un nouveau couple de la forme $(n, n + 1)$ en sélectionnant n grâce à la fonction fst , et à la fonction succ que nous avons déjà définie.

Il faut donc voir cette représentation de cette manière :



Cette représentation nous permet enfin d'écrire le lambda-terme pred :

$$\text{pred} = \lambda n. \text{snd} \ (n \ s' \ z')$$

Vérifions que le résultat de pred sur un entier de Church donne bien l'entier de Church précédent. On suppose donc que n est de la forme

$$\lambda S. \lambda z. \underbrace{S \ (S \ \dots \ (S \ z))}_{n \text{ fois}}$$

Alors le calcul de $(n \ s' \ z')$ applique n fois s' à (z') , ce qui renvoie donc un couple $(n, n - 1)$ selon la définition de s' et z' . Enfin, on applique snd ce qui redonne $n - 1$ sous forme d'entier de Church !

Exemple Calcul détaillé :

$$\begin{aligned}
 \text{pred } 2 &= \underline{(\lambda n. \text{snd } (n \text{ } s' \text{ } z')) \text{ } 2} \\
 &\longrightarrow \text{snd } (2 \text{ } s' \text{ } z') \\
 &= \text{snd } \left((\lambda S. \lambda z. S(S \text{ } z)) \text{ } s' \text{ } z' \right) \\
 &\longrightarrow^* \text{snd } (s' (s' \text{ } z')) \\
 \text{Or, séparément : } s' \text{ } z' &= \underline{(\lambda n. \text{tuple}_2 (\text{succ } (\text{fst } n)) (\text{fst } n)) (\text{tuple}_2 \text{ } 0 \text{ } 0)} \\
 &\longrightarrow \text{tuple}_2 \left(\text{succ } (\text{fst } (\text{tuple}_2 \text{ } 0 \text{ } 0)) \right) (\text{fst } (\text{tuple}_2 \text{ } 0 \text{ } 0)) \\
 &\longrightarrow^* \text{tuple}_2 (\text{succ } 0) \text{ } 0 \\
 &\longrightarrow^* \text{tuple}_2 \text{ } 1 \text{ } 0 \equiv_{\beta} (1, 0) \\
 \text{Similairement : } s' (s' \text{ } z') &\equiv_{\beta} s' (1, 0) \equiv_{\beta} (2, 1) \\
 \text{Ce qui donne enfin : } \text{pred } 2 &\equiv_{\beta} \text{snd } (2, 1) \equiv_{\beta} 1
 \end{aligned}$$

On a bien calculé le prédécesseur de 2, c'est 1

Et grâce à cela, on peut définir la soustraction.

$$\text{sub} = \lambda n. \lambda m. m \text{ pred } n$$

Ce λ -terme calcule bien $n - m$ sous condition que $n \geq m$. En effet, puisqu'on a essentiellement imposé que $\text{pred } 0 \equiv_{\beta} 0$, cela a donné que si $m < n$, on aura $\text{sub } n \text{ } m \equiv_{\beta} 0$.

Ceci concept s'applique aussi aux listes finies en construisant des couples qui contiennent la liste, et sa queue (voire même sa tête). Cette-fois ci, la fonction `pred` devient « `tail` ». Et similairement au fait que le prédécesseur de 0 soit 0, la queue de la liste vide est la liste vide.

2.5 Récursivité avec des points fixes

Jusque là, nous avons vu comment représenter plein de types d'objets différents avec des λ -termes. Mais quelque chose nous embête encore : on ne peut pas faire de boucles, ni de récursivité, ce qui fait que, à ce que l'on sache, le λ -calcul n'est pas Turing-complet.

Dans cette partie, on raisonne sur la récursivité, ce qui permettra de faire des boucles. Il faut que l'on comprenne comment fonctionne la récursive sur le plan calculatoire. Regardons de plus près en OCaml ce qu'est une fonction récursive :

```
let rec f x = if x <= 0 then () else f (x - 1)
```

Le mot-clé `rec` permet d'accéder à la variable `f` dans sa propre définition. Voilà déjà une différence avec le lambda-calcul : on ne peut pas faire référence à la λ -abstraction dans laquelle on se situe, parce qu'une fois qu'elle est appliquée, le lambda et le paramètre disparaissent, ce qui empêche toute sorte de récursivité.

Si on veut espérer pouvoir faire des appels récursifs, il faut donc passer la fonction `f` en argument dans elle-même. Tentons donc de reproduire cela en OCaml (et donc, sans utiliser le mot-clé `rec`) :

```
let f_rec self x = if x <= 0 then () else self (x - 1)
```

Le type de cette fonction est

$$\underbrace{(\text{int} \rightarrow \text{unit})}_{\text{self}} \rightarrow \underbrace{\text{int} \rightarrow \text{unit}}_{\text{f}}$$

C'est très prometteur ! Le type de `self` semble être du même type que `f` elle-même. Ainsi, il ne reste plus qu'à appliquer `f_rec` à elle-même pour remplacer `self` par `f_rec`, ce qui donnera bien un appel récursif à la même fonction :

```
let f = f_rec f_rec
```

Aïe ! – ça n’a pas fonctionné... voici l’erreur émise par OCaml sur le segment de code en rouge :

```
This expression has type (int -> unit) -> int -> unit
but an expression was expected of type int -> unit
Type int -> unit is not compatible with type int
```

C’est une erreur de typage... bizarre. Rappelons-nous qu’il n’est pas possible d’appliquer une fonction à elle-même en OCaml. Et oui, si une telle fonction $h : \alpha \rightarrow \beta$ était appelée à elle-même, ceci imposerait que $\alpha = \alpha \rightarrow \beta$ ce qui n’est pas possible, car autrement la fonction h est de type

$$(\alpha \rightarrow \beta) \rightarrow \beta$$

mais encore

$$(((\dots) \rightarrow \beta) \rightarrow \beta) \rightarrow \beta$$

bref, un type infini, ce qui n’est pas autorisé par définition des types (nous y reviendrons plus tard).

Remarque Dans certains langages de programmation, de tels types sont autorisés (par exemple dans le cadre du sous-typage). On notera alors le type de la fonction h comme ceci :

$$h : \mu x. x \rightarrow \beta$$

Le problème qui fait qu’on ne peut pas « simplifier » l’écriture d’une fonction récursive en OCaml découle du fait que le langage soit fortement typé. Mais, nous n’avons pas encore introduit le typage sur (λ) , et donc il est toujours parfaitement acceptable d’écrire

$$\Delta = \lambda x. x x$$

Et ce genre d’écriture a parfois de drôles de comportements. Par exemple, lorsque nous avons introduit Δ , nous avons vu que le terme $\Delta \Delta$ n’avait pas de forme normale, car une β -réduction redonnait exactement le même terme (et donc qu’il divergeait).

$$\Delta \Delta \longrightarrow \Delta$$

Le λ -terme Δ est appelé « le combinateur divergent ». Pourrait-on utiliser un lambda-terme similaire pour implémenter les fonctions récursives ? Il se trouve que oui ! Le combinateur Δ a une généralisation utile justement pour cela :

Définition

$$Y = \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$$

Ce λ -terme est appelé « combinateur paradoxal » (dû aux travaux de Haskell Curry sur la logique).

Il est souvent noté Y dans la littérature, mais aussi parfois noté fix (en raison d’une propriété que nous allons voir dans un instant).

Le terme fix ressemble un peu au combinateur divergent Δ , et il est difficile de comprendre ce qu’il fait lorsque l’on applique à un terme f . Mais il vérifie une propriété très intéressante qui nous permettra de faire marcher les fonction récursives de la manière que nous avons essayé avec une fonction OCaml.

Théorème Y est un *combinateur de point fixe*. Autrement dit, pour tout terme f :

$$f (Y f) \equiv_{\beta} Y f$$

C'est-à-dire, $Y f$ est un point fixe de f .

Preuve On vérifie manuellement le résultat par β -réduction :

$$\begin{aligned} Y f &= (\lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))) f \\ &\longrightarrow (\lambda x. f (x x)) (\lambda x. f (x x)) \\ &\longrightarrow f (\underbrace{(\lambda x. f (x x)) (\lambda x. f (x x))}_{Y f}) \\ &= f (Y f) \end{aligned}$$

d'où $f (Y f) \equiv_{\beta} Y f$.

Quel intérêt ce terme a-t-il ? Lorsque que j'ai un terme de la forme $Y f$, alors il est en réalité équivalent à $f (Y f)$, mais donc aussi à $f (f (Y f))$, donc à $f (f (f (Y f)))$, et en continuant un nombre arbitraire de fois :

$$Y f \equiv_{\beta} \underbrace{f (f (\dots (f (f (Y f))))))}_{\text{répétition de } f}$$

On observe alors qu'utiliser $Y f$ revient à directement à appeler f avec $Y f$ en paramètre pour un usage prochain dans la fonction f , qui lui aura aussi le même effet, etc... C'est exactement le principe de la récursivité !

Ainsi, si je veux définir une fonction récursive f , je vais d'abord définir une fonction f_{Rec} non-récursive qui se prendra *d'abord en argument* « elle-même », qu'on désignera par *Self*, puis qui prendra les mêmes arguments que f . f_{Rec} utilisera *Self* pour s'appeler elle-même grâce à Y , car on posera à la fin :

$$f = Y f_{\text{Rec}}$$

Tentons cela ! Pour commencer, un exemple simple : on va écrire un lambda-terme qui agit comme une fonction qui fait de décompte à partir d'un entier n de Church jusqu'à 0. Appelons-là *Countdown*.

Remarque Avant de commencer, je me donne temporairement des lambda-termes que je vais utiliser pour coder cette fonction.

- Une constante « unit » qui représente le tuple vide.
- Un λ -terme *IsZero* qui prend un entrée un entier de Church et renvoie un booléen de Church, valant *true* si l'entier était nul, *false* sinon. Ce lambda-terme n'est pas magique, il est défini par :

$$\lambda n. n (\lambda _ . \text{false}) \text{true}$$

1. D'abord, on définit une fonction *Countdown_{Rec}* qui prend une fonction *Self* en argument, puis l'entier initial

$$\text{Countdown}_{\text{Rec}} = \lambda \text{Self}. \lambda n. (???)$$

2. On peut alors écrire le corps de la fonction en vérifiant grâce à *if* si l'entier est nul. Dans ce cas, je retourne *unit*, sinon, je recommence avec $n - 1$ avec un appel récursif.

$$\text{Countdown}_{\text{Rec}} = \lambda \text{Self}. \lambda n. \text{if} (\text{IsZero } n) \text{unit} (\text{Self } (\text{pred } n))$$

3. J'achève la définition de mon lambda-terme *Countdown* avec le combinateur de point fixe :

$$\text{Countdown} = Y \text{Countdown}_{\text{Rec}}$$

4. Vérifions donc l'exécution de ce terme avec 5 comme entrée initiale !

$$\begin{aligned}
\text{Countdown } 5 &= (Y \text{ Countdown}_{\text{Rec}}) \ 5 \\
&\equiv_{\beta} (\text{Countdown}_{\text{Rec}} (Y \text{ Countdown}_{\text{Rec}})) \ 5 \\
&= \text{Countdown}_{\text{Rec}} \text{ Countdown } 5 \\
&= (\lambda \text{Self}. \lambda n. \text{if } (\text{IsZero } n) \ \text{unit} \ (\text{Self } (\text{pred } n))) \ \text{Countdown } 5 \\
&\longrightarrow^* \text{if } (\text{IsZero } 5) \ \text{unit} \ (\text{Countdown } (\text{pred } 5)) \\
&\longrightarrow^* \text{if } (\text{IsZero } 5) \ \text{unit} \ (\text{Countdown } 4) \\
&\longrightarrow^* \text{if } \text{false} \ \text{unit} \ (\text{Countdown } 4) \\
&\equiv_{\beta} \text{Countdown } 4 \\
&\equiv_{\beta} \text{if } (\text{IsZero } 4) \ \text{unit} \ (\text{Countdown } 3) \\
&\equiv_{\beta} \text{Countdown } 3 \\
&\equiv_{\beta} \text{Countdown } 2 \\
&\equiv_{\beta} \text{Countdown } 1 \\
&\equiv_{\beta} \text{Countdown } 0 \\
&\longrightarrow^* \text{if } (\text{IsZero } 0) \ \text{unit} \ (\text{Countdown } 0) \\
&\longrightarrow^* \text{if } \text{true} \ \text{unit} \ (\text{Countdown } 0) \\
&\longrightarrow^* \text{unit}
\end{aligned}$$

Cela a bien donné le décompte de 5 à 0, puis unit. C'est exactement ce que l'on attendait !

Exemple Un autre exemple plus compliqué cette fois-ci : on va définir la fonction factorielle de manière récursive. On l'appelle fact

$$\text{fact}_{\text{Rec}} = \lambda \text{Self}. \lambda n. \text{if } (\text{IsZero } n) \ 1 \ (\text{mul } n \ (\text{Self } (\text{pred } n)))$$

Puis on achève avec :

$$\text{fact} = Y \text{ fact}_{\text{Rec}}$$

Essayons avec 6 :

$$\begin{aligned}
\text{fact } 6 &\equiv_{\beta} \text{fact}_{\text{Rec}} \text{ fact } 6 \\
&\longrightarrow^* \text{if } (\text{IsZero } 6) \ 1 \ (\text{mul } 6 \ (\text{fact } (\text{pred } 6))) \\
&\longrightarrow^* \text{if } (\text{IsZero } 6) \ 1 \ (\text{mul } 6 \ (\text{fact } 5)) \\
&\longrightarrow^* \text{if } \text{false} \ 1 \ (\text{mul } 6 \ (\text{fact } 5)) \\
&\longrightarrow^* \text{mul } 6 \ (\text{fact } 5) \\
&\equiv_{\beta} \text{mul } 6 \ (\text{mul } 5 \ (\text{fact } 4)) \\
&\equiv_{\beta} \text{mul } 6 \ (\text{mul } 5 \ (\text{mul } 4 \ (\text{fact } 3))) \\
&\equiv_{\beta} \text{mul } 6 \ (\text{mul } 5 \ (\text{mul } 4 \ (\text{mul } 3 \ (\text{fact } 2)))) \\
&\equiv_{\beta} \text{mul } 6 \ (\text{mul } 5 \ (\text{mul } 4 \ (\text{mul } 3 \ (\text{mul } 2 \ (\text{fact } 1))))) \\
&\equiv_{\beta} \text{mul } 6 \ (\text{mul } 5 \ (\text{mul } 4 \ (\text{mul } 3 \ (\text{mul } 2 \ (\text{mul } 1 \ (\text{fact } 0))))) \\
&\equiv_{\beta} \text{mul } 6 \ (\text{mul } 5 \ (\text{mul } 4 \ (\text{mul } 3 \ (\text{mul } 2 \ (\text{mul } 1 \ 1))))) \\
&\longrightarrow^* 720
\end{aligned}$$

Exemple Fibonacci récursif

```
fibRec = λSelf. λn.  
  if (IsZero n) 0 (  
    if (IsZero (pred n)) 1 (  
      add (Self (pred n)) (Self (pred (pred n)))  
    )  
  )  
  )  
  
fib = Y fibRec
```

3 Le lambda-calcul simplement typé

Après avoir étudié de lambda-calcul de manière plus ou moins détaillée, il est grand temps de faire le rapprochement avec les langages de programmation et le typage. Ceci va nous prendre quelques temps car il va falloir redéfinir une nouvelle syntaxe pour les types, et tous les autres outils que l'on va utiliser pour le typage.

La première chose que l'on va faire est de choisir une stratégie de réduction qui correspond à l'évaluation dans la plupart des langages de programmation (notamment OCaml). En OCaml, lorsqu'une fonction est appelée, on évalue d'abord tous les arguments qu'elle reçoit, puis on effectue l'application de la fonction. Autrement dit, on ne peut passer dans une fonction que des termes qui ont déjà été totalement évalués : ce sont des valeurs, et non des expressions. Mettons la main sur cette notion :

Définition On définit l'ensemble des valeurs $V \subset \Lambda$ comme ensemble inductif :

$$v := \lambda x. e$$

... où e désigne les lambda-termes quelconques.

Autrement dit, une valeur est une λ -abstraction (une fonction).

Grâce à cette notion de « valeur », on peut détailler la définition de la stratégie que l'on va utiliser. C'est la stratégie CALL-BY-VALUE. Lorsque nous avons défini la relation de β -réduction, nous avons en réalité utilisé la stratégie appelée FULL- β , qui autorise la réduction de n'importe quel redex.

Cependant, la stratégie CALL-BY-VALUE est plus complexe, et il faut imposer des conditions sur un redex pour pouvoir l'évaluer. Pour pouvoir représenter ces conditions, nous allons utiliser, comme en logique propositionnelle, des règles d'inférences.

Définition L'évaluation par CALL-BY-VALUE (appel par valeur) est définie par les règles suivantes :

$$\frac{t_1 \longrightarrow t'_1}{t_1 t_2 \longrightarrow t'_1 t_2} \text{ (E-APP1)}$$

$$\frac{t_2 \longrightarrow t'_2}{v_1 t_2 \longrightarrow v_1 t'_2} \text{ (E-APP2)}$$

$$\frac{}{(\lambda x. e) v \longrightarrow e[x \leftarrow v]} \text{ (E-CALL)}$$

Les deux premières règles permettent tout d'abord d'évaluer les arguments à un λ -terme dans l'ordre. En effet, la première évalue la fonction jusqu'à ce qu'elle soit une valeur. Ensuite, la deuxième règle évalue les arguments jusqu'à ce qu'ils deviennent des valeurs. Enfin, la dernière et plus importante des règles, détecte lorsqu'il y a une lambda-abstraction appliqué à une valeur, et performe la réduction du redex.

Remarquons un autre point important : les variables libres ne sont pas des valeurs. En effet, si je ne sais pas

ce qu'est une variable, je peux pas l'évaluer. On va donc travailler sur un sous-ensemble de termes qui vérifient une propriété qui empêche d'être bloqué par des variables libres.

Définition Un terme $e \in \Lambda$ est dit « fermé » s'il ne contient aucune variable libre : $FV(e) = \emptyset$

Exemple Juste pour bien comprendre, regardons l'évaluation du terme suivant, avec $y \in V$:

$$\begin{aligned}
 \text{app id (id } y) &\equiv_{\beta} (\lambda f. \lambda x. f x) (\lambda t. t) ((\lambda t. t) y) \\
 (1) &\longrightarrow (\lambda x. (\lambda t. t) x) ((\lambda t. t) y) && \text{par (E-APP1) puis (E-CALL)} \\
 (2) &\longrightarrow (\lambda x. (\lambda t. t) x) y && \text{par (E-APP2) puis (E-CALL)} \\
 (3) &\longrightarrow (\lambda t. t) y && \text{par (E-CALL)} \\
 (4) &\longrightarrow y \in V && \text{par (E-CALL)}
 \end{aligned}$$

A chaque étape de cette évaluation (lorsqu'il y a une flèche à gauche d'un terme), il se cache un arbre d'évaluation terminé :

$$\begin{aligned}
 1. & \frac{\frac{}{(\lambda f. \lambda x. f x) (\lambda t. t) \longrightarrow (\lambda x. (\lambda t. t) x)} \text{ (E-CALL)}}{(\lambda f. \lambda x. f x) (\lambda t. t) ((\lambda t. t) y) \longrightarrow (\lambda x. (\lambda t. t) x) ((\lambda t. t) y)} \text{ (E-APP1)} \\
 2. & \frac{\frac{}{((\lambda t. t) y) \longrightarrow y} \text{ (E-CALL)}}{(\lambda x. (\lambda t. t) x) ((\lambda t. t) y) \longrightarrow (\lambda x. (\lambda t. t) x) y} \text{ (E-APP2)} \\
 3. & \frac{}{(\lambda x. (\lambda t. t) x) y \longrightarrow (\lambda t. t) y} \text{ (E-CALL)} \\
 4. & \frac{}{(\lambda t. t) y \longrightarrow y} \text{ (E-CALL)}
 \end{aligned}$$

Ce qui prouve enfin que $\text{app id (id } y) \equiv_{\beta} y$

Avec tout cela en place, il est logique de vouloir imposer que la réduction itérée d'un λ -terme initial soit une valeur $v \in V$. En effet, certains termes, comme $\Delta \Delta$ divergent, ce qui ne modélise pas vraiment une évaluation selon celle d'OCaml.

Le problème se pose donc de la manière suivante : étant donné un terme initial $e \in \Lambda$, **quelle condition suffisante peut-on imposer pour que son évaluation donne une valeur, et ne reste pas bloquée ?**

Spoiler alert : il suffit qu'il soit « bien typé ».

3.1 Extension de (λ) avec des types

Afin de construire le « typage » sur (λ) , il faut tout d'abord définir les types.

Définition Étant donné un ensemble $\mathcal{V}'$ de variables, on définit l'ensemble $\mathcal{T}$ des types par :

$$\begin{aligned}
 \tau &:= x \\
 &| \tau \rightarrow \tau
 \end{aligned}$$

Ainsi, un type est soit une variable $\alpha \in \mathcal{V}'$, soit le type d'une fonction d'un type τ_1 vers un type τ_2 , qui sera noté $\tau_1 \rightarrow \tau_2$. Généralement, on utilise des lettres grecques pour les types, et des lettres de l'alphabet latin pour les variables d'un terme.

On va maintenant étendre la syntaxe du lambda-calcul pour pouvoir utiliser ces types dans des λ -termes.

Définition Le **lambda-calcul simplement typé**, noté $(\lambda_{\rightarrow})$, est l'ensemble des termes $\Lambda_{\rightarrow}$ défini par :

$$\begin{aligned} e &:= x \\ &| \lambda x : \tau. e \\ &| e e \end{aligned}$$

On a juste rajouté une syntaxe pour l'annotation de type d'une variable liée dans une λ -abstraction.

Exemple On peut maintenant traduire certains lambda-termes dans le lambda-calcul simplement typé.

$$\begin{aligned} \text{id} &= \lambda x : \alpha. x \\ \text{app} &= \lambda f : \alpha \rightarrow \beta. \lambda x : \alpha. f x \\ \text{true} &= \lambda x : \alpha. \lambda y : \alpha. y \end{aligned}$$

Cette syntaxe nous a essentiellement permis de « typer » les variables d'une λ -abstraction. Cependant, rien ne nous empêche d'écrire un terme comme celui-ci

$$\lambda f : \tau \rightarrow \tau. f f$$

sans vérifier que $f f$ ne soit « bien typé » lui aussi. Pour l'instant, « bien typé » ne veut pas dire grand chose, donc nous allons le définir.

Peut-être que vous avez déjà entendu parler de la correspondance (ou isomorphisme) de Curry-Howard ! Ce théorème stipule que typer une fonction est rigoureusement équivalent à prouver une formule propositionnelle. Ainsi, comme pour la logique propositionnelle, nous allons nous appuyer sur des arbres de dérivation pour prouver qu'un λ -terme ait un certain type.

Rappelez-vous que lorsque l'on travaille avec des arbres de preuve de formules, on effectue un travail syntaxique sur des *prémisses*, selon des règles d'inférences données. Un séquent est constitué de deux choses :

- un *contexte* noté Γ , qui stocke les formules logiques dont une certaine valuation ν est modèle.
- un *but* noté φ qui est la formule à satisfaire avec cette même valuation (φ est conséquence logique du contexte)

Un tel séquent est donc noté :

$$\Gamma \vdash \varphi$$

Ici, nous allons redéfinir ce qu'est un contexte exactement, puis redéfinir un séquent dans le contexte du typage.

Définition Un contexte Γ est une collection de renseignements sur les types des variables connues. On le définit par induction :

$$\begin{aligned} \Gamma &:= \emptyset \\ &| \Gamma, x : \tau \end{aligned}$$

On peut le voir comme une liste de typages de variables – autrement dit, un élément de $\text{List}(\mathcal{V} \times \mathcal{T})$.

Définition Un séquent est constitué d'un contexte Γ , et du typage d'un lambda-terme $e \in \Lambda_{\rightarrow}$ par un type $\tau \in \mathcal{T}$, qui se note :

$$\Gamma \vdash e : \tau$$

Il ne nous reste plus qu'à définir nos règles d'inférences pour pouvoir construire des arbres de dérivation, ce qui « prouvera » qu'un λ -terme est d'un type τ donné. Cette preuve, comme en logique propositionnelle, sera sous la forme d'un arbre de dérivation terminé, c'est-à-dire que toutes les feuilles n'aient aucun séquent.

Comment choisir nos règles d'inférences ? Et bien, puisque nous avons trois formes de λ -termes différents, nous allons construire trois règles qui nous permettront de typer n'importe quel terme. Détaillons donc comment obtenir ces règles.

- Cas n°1 : e est une variable x . Comment prouver que $x : \tau$? Et bien, la seule manière de le prouver est de vérifier que l'information $x : \tau$ soit bien présente dans le contexte Γ . D'où la première règle d'inférence :

$$\frac{}{\Gamma, x : \tau \vdash x : \tau} \text{ (T-Ax)}$$

Remarque Cette règle d'inférence correspond à la règle (Ax) en logique propositionnelle.

- Cas n°2 : e est une application $f a$. Intuitivement, il faut que la fonction soit d'un type $\alpha \rightarrow \beta$, et que son argument soit de type α , ce qui renverrait donc un terme de type β . Ainsi, pour montrer que $f a : \beta$, on utilise des séquents en hypothèses (appelées « prémisses »), qui montrent que $f : \alpha \rightarrow \beta$, et aussi que $a : \alpha$.

$$\frac{\Gamma \vdash f : \alpha \rightarrow \beta \quad \Gamma \vdash a : \alpha}{\Gamma \vdash f a : \beta} \text{ (T-APP)}$$

Remarque Cette règle correspond à la règle $(\rightarrow_e)$ – le modus ponens.

- Cas n°3 : e est une λ -abstraction de la forme $(\lambda x : \alpha. b)$. Pour déduire le type d'une telle fonction, il suffit d'ajouter $x : \alpha$ dans le contexte, et de déduire le type de $b : \beta$, ce qui donnera au total le type $\alpha \rightarrow \beta$:

$$\frac{\Gamma, x : \alpha \vdash b : \beta}{\Gamma \vdash (\lambda x : \alpha. b) : \alpha \rightarrow \beta} \text{ (T-ABS)}$$

Remarque Cette règle correspond à la règle $(\rightarrow_i)$.

Pour résumer, les règles d'inférence de typage sont donc :

Définition Pour $e \in \Lambda_{\rightarrow}$ et $\tau \in \mathcal{T}$ on prouve que $\Gamma \vdash e : \tau$ avec les règles suivantes :

$$\begin{aligned} & \frac{}{\Gamma, x : \tau \vdash x : \tau} \text{ (T-Ax)} \\ & \frac{\Gamma, x : \alpha \vdash b : \beta}{\Gamma \vdash (\lambda x : \alpha. b) : \alpha \rightarrow \beta} \text{ (T-ABS)} \\ & \frac{\Gamma \vdash f : \alpha \rightarrow \beta \quad \Gamma \vdash a : \alpha}{\Gamma \vdash f a : \beta} \text{ (T-APP)} \end{aligned}$$

Exemple Prouvons que $\text{app} : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$

$$\begin{aligned} & \frac{\frac{\frac{}{f : \alpha \rightarrow \beta, x : \alpha \vdash f : \alpha \rightarrow \beta} \text{ (T-Ax)}}{f : \alpha \rightarrow \beta, x : \alpha \vdash f x : \beta} \text{ (T-APP)}}{f : \alpha \rightarrow \beta \vdash (\lambda x : \alpha. f x) : \alpha \rightarrow \beta} \text{ (T-ABS)} \\ & \frac{}{\vdash (\lambda f : \alpha \rightarrow \beta. \lambda x : \alpha. f x) : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta} \text{ (T-ABS)} \end{aligned}$$

L'arbre de dérivation étant bien terminé, on conclut avec le résultat du typage.

$$\text{app} : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$$

Théorème Inversibilité

Les règles d'inférences sont inversibles. Plus précisément

1. Si $\Gamma \vdash x : \tau$, alors $(x : \tau) \in \Gamma$.
2. Si $\Gamma \vdash (\lambda x : \alpha. b) : \tau$, alors $\tau = \alpha \rightarrow \beta$ (avec un certain $\beta \in \mathcal{T}$), et $\Gamma, x : \alpha \vdash b : \beta$
3. Si $\Gamma \vdash e_1 e_2 : \beta$, alors il existe un type α tel que $\Gamma \vdash e_1 : \alpha \rightarrow \beta$, et $\Gamma \vdash e_2 : \alpha$

Preuve Directe par lecture des règles d'inférences. Il n'y a presque rien à dire.

Ce théorème est utilisé si souvent que ce n'est visiblement parfois pas la peine de le mentionner (ce qui m'a bien embêté lorsque je ne comprenais pas la preuve d'un des lemmes que nous allons montrer).

3.2 Sécurité = avancement + préservation

En quoi ces mécanismes nous permettent-ils de déduire des propriétés d'évaluation sur les lambda-termes simplement typés ? Nous avons défini la relation de typage avec des règles d'inférences, et de même, l'évaluation (sous la stratégie d'appel par valeur) avec d'autres règles d'inférences. De ce fait, nous allons débloquent le pouvoir de faire jouer les deux ensemble pour appliquer des propriétés de typage sur des évaluations, et aussi, appliquer des propriétés de réduction sur le typage.

Le système de type que l'on a défini jusqu'ici vérifie une propriété très basique mais tout autant essentielle : on l'appelle la « *sécurité* » (safety). Cette propriété stipule grossièrement que les termes typés « ne vont pas mal tourner » lorsqu'on les évaluera. Plus précisément, par « mal tourner », on veut entendre qu'un terme ne sera jamais bloqué pendant l'évaluation, et qu'il terminera toujours par une valeur $v \in V$.

Cette propriété de « sécurité » se décompose en deux invariants :

- **l'avancement** : un terme bien typé n'est pas bloqué – c'est soit une valeur, soit c'est un terme qui peut effectuer une étape d'évaluation (selon la stratégie CALL-BY-VALUE).
- **la préservation** : si un terme bien typé effectue une étape d'évaluation, le terme résultant est aussi bien typé.

On voit bien intuitivement que ces invariants montrent qu'un terme bien typé sera à un certain moment évalué en une valeur finale.

Dans cette partie on s'intéresse à la preuve de la sécurité.

3.2.1 Quelques lemmes

Avant d'attaquer la preuve, on va se servir de quelques lemmes qui nous faciliteront certains raisonnements.

Le premier lemme permet d'ajouter des variables dans un contexte sans affecter le résultat du typage.

Lemme Affaiblissement

Soit x une variable telle que $x \notin \Gamma$. Alors

$$\Gamma \vdash t : \tau \implies \Gamma, x : \alpha \vdash t : \tau$$

L'ajout de $x : \alpha$ dans le contexte permet de prouver la même chose sur le type de t .

Si l'on voulait être le plus rigoureux, il faudrait montrer que les preuves de typages sont stables par permutation du contexte. Je passe sous le tapis ce détail qui n'est pas trop intéressant, et je m'autorise à interchanger les variables dans un contexte, ou à extraire des sous-contextes à partir d'un contexte, sans me soucier de l'ordre dans lequel ses variables sont écrites.

Le second va permettre de déduire la forme d'un terme étant donné son type. Puisque notre système de typage est pour l'instant très simple, ce lemme est plutôt court.

Lemme Lemme des formes canoniques

Soit $v \in V$ une valeur de type $\tau_1 \rightarrow \tau_2$. Alors $\exists e \in \Lambda_{\rightarrow} \mid v = \lambda x : \tau_1. e$

Autrement dit, une valeur dont le type est celui d'une abstraction est elle-même une abstraction.

Preuve Si v est une valeur, alors, par définition des valeurs V , v s'écrit comme

$$v = \lambda x : \alpha. e$$

avec $e \in \Lambda_{\rightarrow}$ et $\alpha \in \mathcal{T}$.

De plus, on sait que $v : \tau_1 \rightarrow \tau_2$, donc l'arbre de preuve pour v s'écrit :

$$\frac{\dots}{\Gamma \vdash \underbrace{(\lambda x : \tau_1. e)}_v : \tau_1 \rightarrow \tau_2} \text{ (T-Abs)}$$

ce qui impose $\lambda x : \tau_1. e = \lambda x : \alpha. e$, donc $\alpha = \tau_1$. D'où $v = \lambda x : \tau_1. e$.

Le lemme suivant est plus délicat et va nous permettre de conserver des propriétés de typages après une β -réduction. Pour cela, il faut montrer que l'opération de substitution conserve les types.

Lemme Stabilité par substitution

$$\Gamma, x : \sigma \vdash t : \tau \text{ et } \Gamma \vdash s : \sigma \implies \Gamma \vdash t[x \leftarrow s] : \tau$$

Autrement dit : si en sachant que $x : \sigma$, on peut prouver que $t : \tau$, et qu'on peut aussi prouver que $s : \sigma$, alors on peut prouver aussi que la substitution de x par s dans t est du même type que t .

La preuve est trop grande donc je suis obligé de la sortir de la boîte du théorème.

Preuve : Puisque $\Gamma, x : \sigma \vdash t : \tau$, on a donc un arbre de preuve terminé sur lequel on va effectuer une induction structurale.

- Cas (T-Ax) : Si la première règle utilisée est (T-Ax), alors l'arbre de dérivation est de la forme

$$\frac{}{\Gamma, x : \sigma, \dots, \underline{z} : \tau, \dots \vdash z : \tau} \text{ (T-Ax)}$$

avec $t = z \in \mathcal{V}$ une variable. Lorsque l'on effectue une substitution sur une variable, il y a deux sous-cas :

- $\underline{z} = \underline{x}$: donc $z[x \leftarrow s] = s$. On veut donc avoir $\Gamma \vdash s : \tau$, mais on peut déduire que $\tau = \sigma$ par inversion, ce qui fait partie des hypothèses du lemme. OK.
- $\underline{z} \neq \underline{x}$, donc la substitution ne fait rien, et on veut montrer que $z : \tau$, ce qui déjà prouvé.

- Cas (T-Abs) : on a donc

$$\frac{\dots}{\Gamma, x : \sigma \vdash \underbrace{(\lambda y : \tau_1. e)}_t : \underbrace{\tau_1 \rightarrow \tau_2}_\tau} \text{ avec } \begin{cases} t = \lambda y : \tau_1. e \\ \tau = \tau_1 \rightarrow \tau_2 \end{cases}$$

On va supposer que $x \neq y$ et $y \notin \text{FV}(s)$, sinon on ne pourrait pas effectuer la substitution. Effectuons-là :

$$(\lambda y : \tau_1. e)[x \leftarrow s] = (\lambda y : \tau_1. e[x \leftarrow s])$$

Prouvons donc que $(\lambda y : \tau_1. e[x \leftarrow s]) : \tau_1 \rightarrow \tau_2$.

$$\frac{\frac{???}{\Gamma, y : \tau_1 \vdash e[x \leftarrow s] : \tau_1 \rightarrow \tau_2}}{\Gamma \vdash (\lambda y : \tau_1. e[x \leftarrow s]) : \tau_1 \rightarrow \tau_2} \text{ (T-Abs)}$$

Or, par affaiblissement :

$$\Gamma, x : \sigma \vdash s : \sigma \implies \Gamma, x : \sigma, y : \tau_1 \vdash s : \sigma$$

et puisque

$$\Gamma, x : \sigma, y : \tau_1 \vdash e : \tau_2$$

on peut appliquer l'hypothèse d'induction sur e , on a que

$$\Gamma, y : \tau_1 \vdash e[x \leftarrow s] : \tau_2$$

Ce qui complète l'arbre de preuve de $(\lambda y : \tau_1. e[x \leftarrow s]) : \tau_1 \rightarrow \tau_2$.

- Cas (T-App) : l'arbre s'écrit donc

$$\frac{\frac{\dots}{\Gamma, x : \sigma \vdash t_1 : \tau_1 \rightarrow \tau_2} \quad \frac{\dots}{\Gamma, x : \sigma \vdash t_2 : \tau_1}}{\Gamma, x : \sigma \vdash \underbrace{t_1 t_2}_t : \underbrace{\tau_2}_\tau} \text{ (T-APP)} \quad \text{avec} \quad \begin{cases} t = t_1 t_2 \\ \tau = \tau_2 \\ \tau_1 \in \mathcal{T} \end{cases}$$

La substitution de x par s dans ce terme est $(t_1 t_2)[x \leftarrow s] = (t_1[x \leftarrow s]) (t_2[x \leftarrow s])$. Prouvons alors que le type est le même pour ce nouveau terme : $\Gamma \vdash (t_1[x \leftarrow s]) (t_2[x \leftarrow s]) : \tau_2$

$$\frac{\frac{???}{\Gamma \vdash (t_1[x \leftarrow s]) : \tau_1 \rightarrow \tau_2} \quad \frac{???}{\Gamma \vdash (t_2[x \leftarrow s]) : \tau_1}}{\Gamma \vdash (t_1[x \leftarrow s]) (t_2[x \leftarrow s]) : \tau_2} \text{ (T-APP)}$$

Il faut compléter les feuilles de cet arbre : on va le faire en appliquant l'hypothèse d'induction avec les sous-termes t_1 et t_2 . En effet, les hypothèses pour l'appliquer sont déjà remplies :

$$\begin{cases} \Gamma, x : \sigma \vdash t_1 : \tau_1 \rightarrow \tau_2 \\ \Gamma \vdash s : \sigma \end{cases} \implies \Gamma \vdash (t_1[x \leftarrow s]) : \tau_1 \rightarrow \tau_2$$

$$\begin{cases} \Gamma, x : \sigma \vdash t_2 : \tau_1 \\ \Gamma \vdash s : \sigma \end{cases} \implies \Gamma \vdash (t_2[x \leftarrow s]) : \tau_1$$

Ceci termine l'arbre de dérivation et achève donc la preuve du lemme.

3.2.2 Preuve de la propriété

Grâce à ces lemmes, on peut enfin procéder à la preuve de la sécurité, en prouvant séparément l'avancement et la séparation.

Théorème Avancement

Soit $t \in \Lambda_{\rightarrow}$ un terme fermé et bien typé, c'est-à-dire, $\Gamma \vdash t : \tau$, pour un certain type τ . Alors l'une des deux propositions suivantes est vraie :

- $t \in V$: c'est une valeur ;
- $\exists t' \in \Lambda_{\rightarrow} \mid t \longrightarrow t'$: on peut effectuer une étape de réduction.

Preuve : On effectue une induction sur l'arbre de preuve de l'hypothèse $\Gamma \vdash t : \tau$. Il y a trois cas, mais seulement l'un d'entre eux est vraiment intéressant.

- Cas (T-Ax) : impossible, car t est fermé, donc il ne peut pas être une variable libre.
- Cas (T-Abs) : par inversibilité, on en déduit que $t = \lambda y : \alpha. e$, donc $t \in V$. Il n'y a rien à faire.
- Cas (T-App) : par inversibilité de la règle, on a $t = t_1 t_2$, avec $\Gamma \vdash t_1 : \tau_1 \rightarrow \tau_2$ et $\Gamma \vdash t_2 : \tau_1$.

Ainsi, t_1 et t_2 sont des termes fermés et bien typés. Par hypothèse d'induction, ils peuvent être des valeurs ou peuvent être évalués.

- si $t_1 \notin V$, alors $\exists t'_1 \in \Lambda_{\rightarrow} \mid t_1 \rightarrow t'_1$. Et donc la règle (E-App1) s'applique, ce qui donne

$$t = t_1 t_2 \longrightarrow t'_1 t_2 = t'$$

- si $t_1 \in V$ mais $t_2 \notin V$, alors $\exists t'_2 \in \Lambda_{\rightarrow} \mid t_2 \rightarrow t'_2$ et la règle (E-App2) s'applique donc, et donne

$$t = \underbrace{t_1}_{\in V} t_2 \longrightarrow t_1 t'_2 = t'$$

- si $t_1 \in V$ et $t_2 \in V$, par définition des valeurs, on doit avoir $t_1 = (\lambda x : \alpha. b)$ (pour un certain terme b), et donc la règle (E-CALL) s'applique et donne

$$t = (\lambda x : \alpha. b) t_2 \longrightarrow b[x \leftarrow t_2] = t'$$

Le lemme est désormais démontré.

Théorème Préservation

Le typage est préservé par β -réduction.

$$\left\{ \begin{array}{l} \exists t' \in \Lambda_{\rightarrow} \mid t \longrightarrow t' \\ \Gamma \vdash t : \tau \end{array} \right\} \implies \Gamma \vdash t' : \tau$$

Preuve : Encore une fois par induction, mais cette fois sur l'arbre de dérivation de $t \longrightarrow t'$.

Remarquons que n'importe quelle réduction sur le terme t implique que c'est une application, donc on en déduit que l'arbre de typage de $t : \tau$ commence toujours par la règle (T-App).

- Cas E-App1, donc $t = t_1 t_2$. Les arbres pour l'évaluation et le typage sont donc :

$$\frac{\frac{\dots}{t_1 \longrightarrow t'_1}}{t_1 t_2 \longrightarrow t'_1 t_2} \text{ (E-App1)} \quad \frac{\frac{\dots}{\Gamma \vdash t_1 : \tau_1 \rightarrow \tau_2} \quad \frac{(\star)}{\Gamma \vdash t_2 : \tau_1}}{\Gamma \vdash t_1 t_2 : \tau_2} \text{ (T-App)}$$

Montrons donc que $t'_1 t_2 : \tau_2$ avec un arbre terminé :

$$\frac{\frac{(\text{h.i})}{\Gamma \vdash t'_1 : \tau_1 \rightarrow \tau_2} \quad \frac{(\star)}{\Gamma \vdash t_2 : \tau_1}}{\Gamma \vdash t'_1 t_2 : \tau_2} \text{ (T-App)}$$

On obtient l'arbre de typage (h.i) avec l'hypothèse d'induction. Puisque $t_1 \longrightarrow t'_1$ et que $\Gamma \vdash t_1 : \tau_1 \rightarrow \tau_1$, alors $\Gamma \vdash t'_1 : \tau_1 \rightarrow \tau_1$ est prouvé par un arbre terminé qui sera donc (h.i). OK.

- Cas E-App2, on a $t = v_1 t_2$, avec $v_1 \in V$. Les arbres pour l'évaluation et le typage sont donc :

$$\frac{\frac{\dots}{t_2 \longrightarrow t'_2}}{v_1 t_2 \longrightarrow v_1 t'_2} \text{ (E-App2)} \quad \frac{\frac{(\star)}{\Gamma \vdash v_1 : \tau_1 \rightarrow \tau_2} \quad \frac{\dots}{\Gamma \vdash t_2 : \tau_1}}{\Gamma \vdash v_1 t_2 : \tau_2} \text{ (T-App)}$$

Montrons donc que $v_1 t'_2 : \tau_2$ avec un arbre terminé :

$$\frac{\frac{(\star)}{\Gamma \vdash v_1 : \tau_1 \rightarrow \tau_2} \quad \frac{(\text{h.i})}{\Gamma \vdash t'_2 : \tau_1}}{\Gamma \vdash v_1 t'_2 : \tau_2} \text{ (T-APP)}$$

Similairement au cas précédent, puisque $t_2 \longrightarrow t'_2$ et que $\Gamma \vdash t_2 : \tau_1$, alors $\Gamma \vdash t'_2 : \tau_1$ est prouvé par un arbre nommé (h.i). OK.

- **Cas E-CALL.** Ce cas est plus délicat.

On doit avoir $t = (\lambda x : \tau_1. e) v_2$, avec $v_2 \in V$. Pour avoir le plus d'information possible, il faut davantage développer l'arbre de typage de ce terme. Les arbres sont donc :

$$\frac{\overline{(\lambda x : \tau_1. e) v_2 \longrightarrow e[x \leftarrow v_2]}}{\text{ (E-CALL)}} \quad \frac{\frac{\overline{\dots}}{\Gamma, x : \tau_1 \vdash e : \tau_2} \text{ (T-ABS)} \quad \frac{\overline{\dots}}{\Gamma \vdash v_2 : \tau_1} \text{ (T-APP)}}{\Gamma \vdash (\lambda x : \tau_1. e) v_2 : \tau_2}$$

On ne peut pas utiliser l'hypothèse d'induction ici (sinon la preuve serait fausse, car il n'existerait aucun cas de base). Mais rappelons-nous qu'il faut montrer que $\Gamma \vdash e[x \leftarrow v_2] : \tau_2$. On peut appliquer le lemme de stabilité par substitution !

$$\left\{ \begin{array}{l} \Gamma, x : \tau_1 \vdash e : \tau_2 \\ \Gamma \vdash v_2 : \tau_1 \end{array} \right\} \implies \Gamma \vdash e[x \leftarrow v_2] : \tau_2$$

Cela montre donc le résultat de la préservation. On a bien géré tous les cas par induction donc le théorème est prouvé.

Conclusion : on a réussi à montrer la sécurité en montrant la préservation et l'avancement. Ainsi, tous les termes fermes bien-typés sont soit évalués en d'autres termes bien-typés, soit ce sont des valeurs ! Ceci prouve formellement que le typage permet à l'évaluation du lambda-calcul de ne jamais rester bloquée, et par extension, les langages de programmation.

3.3 Simples extensions

A partir de maintenant, nous sommes libres d'étendre notre langage (le lambda-calcul) avec les objets qui nous intéressent : des entiers, des chaînes de caractères, mais aussi des types plus compliqués. Cela est justifié à condition d'étendre les preuves des lemmes et des théorèmes de la partie précédente. Nous n'allons pas tout redémontrer, mais plutôt visiter de plus près le monde des types tels que vous les connaissez en OCaml (et d'autres langages bien sûr).

3.3.1 Une « réparation » de la récursivité

En se limitant au termes bien typés, j'ai cassé la récursivité. En effet, comme en OCaml, le terme $x x$ est mal-typé (n'admet pas de preuve de typage), et par extension, le combinateur paradoxal Y est mal typé. Ceci m'empêche d'écrire des fonction récursives bien typées.

En vérité, OCaml ne fonctionne pas en arrière plan avec un tel combinateur paradoxal, mais plutôt, lorsqu'un appel récursif est effectué, il s'agit en réalité d'un appel usuel à une fonction, qui est une adresse dans la mémoire. Il se trouve juste que l'adresse en question est celle de la fonction qui est en train d'être définie.

Pour régler ce problème, on triche en se donnant une terme magique Y_* qui agit comme le combinateur paradoxal de Curry, sans se soucier de sa définition. Ainsi, avec une fonction pré-récursive qui se prend elle-même en argument

$$f_{\text{Rec}} : \underbrace{(\alpha_1 \rightarrow \alpha_2 \rightarrow \dots \rightarrow \alpha_n \rightarrow \beta)}_{\text{Self}} \rightarrow \underbrace{\alpha_1 \rightarrow \alpha_2 \rightarrow \dots \rightarrow \alpha_n \rightarrow \beta}_f$$

...on s'autorise à définir $Y_* f_{\text{Rec}} = f : \alpha_1 \rightarrow \alpha_2 \rightarrow \dots \rightarrow \alpha_n \rightarrow \beta$ qui agit comme une fonction récursive (dont l'exécution est dictée par f_{Rec}).

3.3.2 Entiers, booléens, etc.

Similairement à la récursivité, OCaml ne fonctionne pas avec des représentation de Church pour ses entiers, ni pour ses booléens.

On pourrait poser un nom spécial pour désigner le type qu'ont les lambda-termes 0, 1, ..., qui représentent les entiers de Church :

$$\text{Nat} = (\omega \rightarrow \omega) \rightarrow \omega \rightarrow \omega$$

...mais à ce moment là, on arrive à types les fonctions comme succ, add, mais pas mul, ni exp, tetra, penta, etc.

De même pour les booléens, on pourrait imaginer un type qui marche pour les termes true et false :

$$\text{Bool} = (\omega \rightarrow \omega) \rightarrow \omega \rightarrow \omega$$

...et – aïe – similairement, je ne peux plus typer and, or, not, xor, etc.

Remarque Il existe un système de type plus général (appelée « System F ») qui type le lambda calcul polymorphe, dans lequel on autorise les lambda-termes à prendre des types en arguments, ce qui donne existence au types quantifiés :

$$\text{id} = \Lambda\chi. \lambda x : \chi. x \quad : \quad \forall\chi, \chi \rightarrow \chi$$

On peut lire : « id est de type $\chi \rightarrow \chi$ pour n'importe quel χ ».

Un autre problème est que les entiers d'OCaml représentent les entiers relatifs, ce que nous n'avons pas vraiment défini dans (λ).

Pour ne pas s'embêter avec toutes ces considérations, on peut se permettre d'enrichir le λ -calcul avec des nouveaux objets « externes » qui auront leur propres règles d'évaluation. On pourra appeler ce nouveau système de calcul ($\Lambda_{\rightarrow}^{\text{NB}}$).

Ici, faisons-le avec les entiers relatifs, et les booléens. D'abord, on commence par redéfinir l'ensemble $\Lambda_{\rightarrow}^{\text{NB}}$ des termes :

$e :=$	
x	(variables)
$\lambda x. e$	(abstractions)
$e e$	(applications)
false true	(booléens)
if e then e else e	(tests)
$e \wedge e$ $e \vee e$ $e \oplus e$ $\neg e$	(opérations booléennes)
n	(entiers relatifs)
$e + e$ $e - e$ $e \times e$ e / e $-e$	(opération arithmétiques)
$e = e$ $e \leq e$ $e < e$ $e \geq e$ $e > e$	(tests entiers)

Cette nouvelle syntaxe nous permet maintenant de définir des termes plus sophistiqués qui ressemblent un peu plus à des fonctions dans un langage de programmation : $\lambda x. \text{if } x \leq 0 \text{ then } 0 \text{ else } ((512 \times x) / (x \times x))$

Il faut aussi redéfinir les valeurs V :

$v :=$	
$\lambda x. e$	(abstractions)
false true	(booléens)
n	(entiers relatifs)

Ensuite, les nouvelles règles d'inférences pour l'évaluation. On garde les mêmes que pour $\Lambda_{\rightarrow}$:

$$\frac{t_1 \longrightarrow t'_1}{t_1 t_2 \longrightarrow t'_1 t_2} \text{ (E-APP1)} \quad \frac{t_2 \longrightarrow t'_2}{v_1 t_2 \longrightarrow v_1 t'_2} \text{ (E-APP2)} \quad \frac{}{(\lambda x. e) v_2 \longrightarrow e[x \leftarrow v_2]} \text{ (E-CALL)}$$

Puis on rajoute ce qui concerne les booléens :

$$\frac{}{\text{if true then } e_1 \text{ else } e_2 \longrightarrow e_1} \text{ (E-IfTrue)} \quad \frac{}{\text{if false then } e_1 \text{ else } e_2 \longrightarrow e_2} \text{ (E-IfFalse)}$$

$$\frac{g \longrightarrow g'}{\text{if } g \text{ then } e_1 \text{ else } e_2 \longrightarrow \text{if } g' \text{ then } e_1 \text{ else } e_2} \text{ (E-If)}$$

Je ne vais pas écrire toutes les règles pour toutes les opérations, mais juste une fois pour une opération quelconque $\diamond$ (soit sur les booléens, soit sur les entiers). Le principe reste le même pour les opérateurs unaires.

$$\frac{a \longrightarrow a'}{a \diamond b \longrightarrow a' \diamond b} \text{ (E-}\diamond\text{Left)} \quad \frac{b \longrightarrow b'}{v \diamond b \longrightarrow v \diamond b'} \text{ (E-}\diamond\text{Right)}$$

$$\frac{v_1 \diamond v_2 = R}{v_1 \diamond v_2 \longrightarrow R} \text{ (E-}\diamond\text{)}$$

En particulier, la dernière règle a une prémisse qui n'est pas un séquent. Elle est nécessaire pour empêcher l'évaluation de true + false, ou $\neg 1$ par exemple... On peut le voir comme si la règle avait été dupliquée, pour chaque opérateur et pour chaque cas possible.

Enfin, les types. On redéfinit les types comme ceci :

$$\tau :=$$

- | x (types variables)
- | $\tau \rightarrow \tau$ (fonctions)
- | Bool (booléens)
- | Int (entiers naturels)

Puis les règles usuelles :

$$\frac{}{\Gamma, x : \tau \vdash x : \tau} \text{ (T-Ax)} \quad \frac{\Gamma, x : \alpha \vdash b : \beta}{\Gamma \vdash (\lambda x : \alpha. b) : \alpha \rightarrow \beta} \text{ (T-Abs)} \quad \frac{\Gamma \vdash f : \alpha \rightarrow \beta \quad \Gamma \vdash a : \alpha}{\Gamma \vdash f a : \beta} \text{ (T-App)}$$

Ensuite, le type d'un test : il faut vérifier que la condition soit de type Bool, et que les deux résultats possible soient du même types.

$$\frac{\Gamma \vdash g : \text{Bool} \quad \Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash (\text{if } g \text{ then } e_1 \text{ else } e_2) : \tau} \text{ (T-If)}$$

Pour toute opération

- $\diamond_{\text{Bool}}$ binaire sur les booléens
- $*_{\text{Bool}}$ unaire sur les booléens
- $\diamond_{\text{Int}}$ binaire sur les entiers
- $*_{\text{Int}}$ unaire sur les booléens

on ajoute les règles :

$$\frac{\Gamma \vdash a : \text{Bool} \quad \Gamma \vdash b : \text{Bool}}{\Gamma \vdash a \diamond_{\text{Bool}} b : \text{Bool}} \text{ (T-}\diamond_{\text{Bool}}\text{)} \quad \frac{\Gamma \vdash a : \text{Int} \quad \Gamma \vdash b : \text{Int}}{\Gamma \vdash a \diamond_{\text{Int}} b : \text{Int}} \text{ (T-}\diamond_{\text{Int}}\text{)}$$

$$\frac{\Gamma \vdash x : \text{Bool}}{\Gamma \vdash *_{\text{Bool}} x : \text{Bool}} \text{ (T-}\ast_{\text{Bool}}\text{)} \quad \frac{\Gamma \vdash x : \text{Int}}{\Gamma \vdash *_{\text{Int}} x : \text{Int}} \text{ (T-}\ast_{\text{Int}}\text{)}$$

Cela devrait suffire pour un langage basé sur le λ -calcul. Il faut maintenant prouver la sécurité. Alors c'est parti, allez-y !

3.3.3 Types algébriques

On peut encore étendre le monde des types. C'est un ensemble inductif, donc il est construit avec des constructeurs, tous ayant une certaine arité positive ou nulle. Par exemple, Bool et Int sont des constructeurs d'arité 0, tandis que $\rightarrow$ est un constructeur d'arité 2 car il prend deux types et en fait un nouveau. On peut le voir comme une application $\cdot \rightarrow \cdot : \mathcal{T} \times \mathcal{T} \rightarrow \mathcal{T}$.

Pourquoi peut-on parler de types algébriques ? Lorsque l'on emploie ce terme, on fait référence aux deux opérations suivantes entre les types : $+$ et $\times$.

Produit. Vous connaissez déjà $\times$: c'est le produit cartésien. Si on se donne deux types α et β , alors on note $\alpha \times \beta$ le type des couples $(a : \alpha, b : \beta)$. De manière générale, on notera $\alpha_1 \times \dots \times \alpha_n$ le type des n -uplets dont les éléments sont de la forme $(a_1 : \alpha_1, \dots, a_n : \alpha_n)$.

Remarque Attention ! $\alpha \times \beta \times \gamma \neq (\alpha \times \beta) \times \gamma \neq \alpha \times (\beta \times \gamma)$.

En effet, formellement, $(a, b, c) \neq ((a, b), c) \neq (a, (b, c))$.

On étend rapidement le langage (idem pour les valeurs V) :

$$\begin{aligned} e := & \dots \\ & | (e, e) \quad (2\text{-uplets}) \\ & | (e, e, e) \quad (3\text{-uplets}) \\ & | \dots \end{aligned}$$

La règle de typage pour les n -uplets est, logiquement ($\forall n \geq 2$):

$$\frac{\Gamma \vdash a_1 : \alpha_1 \quad \dots \quad \Gamma \vdash a_n : \alpha_n}{\Gamma \vdash (a_1, \dots, a_n) : \alpha_1 \times \dots \times \alpha_n} \text{ (T-TUPLE}_n\text{)}$$

Somme. Vous connaissez aussi la somme entre deux types, mais vous ne vous en rendez peut-être pas compte. La somme $\alpha + \beta$ correspond au type des valeurs qui sont de type α , soit de type β . Par exemple, $\text{Int} + \text{Bool}$ est le type des valeurs qui sont soit un entier, soit un booléen.

Pour étendre le langage, il faut se demander : comment noter une valeur $x : \alpha + \beta$? Il faut se donner une syntaxe différente des valeurs de types de α et de celles de type β , sinon on ne pourra pas faire la différence ! Encore plus contraignant, on doit pouvoir faire la différence entre les valeurs de type $\alpha + \beta$, et de celles de type $\alpha + \gamma$...

Cela semble bien compliqué, donc plutôt, on va laisser l'utilisateur donner lui-même un nom au type somme qu'il veut utiliser. Je m'explique. En OCaml, vous avez accès à la syntaxe suivante :

$$\text{type } \Sigma = \text{Var}_1 \text{ of } \tau_1 \mid \dots \mid \text{Var}_n \text{ of } \tau_n \ ;;$$

Et en effet, les valeurs $x : \Sigma$ sont bien des valeurs de types $\tau_1 + \dots + \tau_n$, car cette construction OCaml impose que x ne soit qu'un seul des variants parmi $\text{Var}_1, \dots, \text{Var}_n$.

Ainsi, pour n'importe quel type somme Σ défini comme la somme de variants $\text{Var}_1, \dots, \text{Var}_n$, associés aux types $\tau_1, \dots, \tau_n$, on ajoute le terme (et la valeur) :

$$\begin{aligned} e := & \dots \\ & | \text{Var}_1 e \quad (\text{variant } \text{Var}_1 \text{ du type } \Sigma) \\ & | \dots \\ & | \text{Var}_n e \quad (\text{variant } \text{Var}_n \text{ du type } \Sigma) \end{aligned}$$

et comme pour les tuples, il est facile d'en déduire les règles d'inférences. $\forall i \in \llbracket 1, n \rrbracket$:

$$\frac{\Gamma \vdash e : \tau_i}{\Gamma \vdash \text{Var}_i e : \Sigma} \text{ (T-Var}_i\text{)}$$

4 Inférence de type en OCaml

A présent, nous sommes munis de tous les outils dont nous avons besoin pour mettre au point un algorithme d'inférence de types. Nous avons étudié le lambda calcul, et nous nous en sommes servi pour prouver que le typage rendait l'évaluation correcte. Dans cette partie, nous allons redécouvrir un algorithme classique qui permet de déduire les types d'une expression OCaml sans que l'utilisateur n'ait à le typer lui-même.

Observons quelque chose : pour typer une expression, nous devons gérer tous les cas possibles de la syntaxe de cette expression, et c'est la raison pourquoi nous avons une règle d'inférence par forme syntaxique. Il est logique de penser qu'un algorithme va donc se baser sur la syntaxe d'un programme (et en fait, il n'y a pas grand chose d'autre sur lequel il peut s'appuyer).

4.1 Définition du langage support

Définissons la syntaxe d'OCaml de la même manière que nous l'avons fait pour le λ -calcul et ses extensions. On ne va pas tout prendre un compte, et seulement travailler sur une sous-partie d'OCaml.

Définition Expressions et motifs d'OCaml

Les expressions e sont définies par

$e :=$	
x	(identificateurs)
$\text{fun } x \rightarrow e$	(abstractions)
$e \ e$	(applications)
$\text{let } p = e \text{ in } e$	(déclarations)
$e ; e$	(séquençage)
$() \mid (e) \mid (e, e) \mid (e, e, e) \mid \dots$	(n -uplets)
$\text{Var}_i \ e \mid \dots$	(variants du type Σ)
$\text{match } e \text{ with } p \rightarrow e \mid \dots$	(filtrage)
$\text{true} \mid \text{false}$	(booléens)
$\text{if } e \text{ then } e \text{ else } e$	(tests)
$e \ \&\& \ e \mid e \ \ e \mid \text{not } e$	(opérations booléennes)
$0 \mid 1 \mid 2 \mid 3 \mid \dots$	(entiers relatifs)
$e + e \mid e - e \mid e * e \mid e / e \mid - e$	(opération arithmétiques)
$e = e \mid e < e \mid e <= e \mid e < e \mid e >= e \mid e > e$	(comparaison)

Les motifs sont les expressions qui apparaissent à gauche du signe égal dans `let`, et aussi à gauche d'une flèche `->` dans un `match` :

$p :=$	
x	(identificateurs)
$() \mid (p) \mid (p, p) \mid (p, p, p) \mid \dots$	(n -uplets)
$\text{Var}_i \ p \mid \dots$	(variants du type Σ)
$\text{true} \mid \text{false}$	(booléens)
$0 \mid 1 \mid 2 \mid 3 \mid \dots$	(entiers relatifs)

Pourquoi avons nous deux catégories d'expressions ? L'une est appelée celle des « expressions », et l'autre celle des « motifs ». La raison pour cela est qu'il faut faire la distinction entre les expressions qui sont utilisées comme valeurs, et celles qui se trouvent à gauche d'un signe « = ». Les motifs (ou « patterns ») ne sont pas des expressions qui représentent des valeurs, mais servent à déconstruire une valeur qui se situe derrière une variable.

```
let (x, y) = calculate_point () in ...
```

Nous n'allons pas définir les règles d'évaluation pour OCaml, car ce n'est pas le but de cette partie. Par contre, nous devons définir celles du typage. D'abord, définissons les types.

Définition Types en OCaml

Les types τ d'OCaml sont définis par la structure suivante :

$\tau :=$	
x	(type variables)
$()$ (τ) (τ, τ) (τ, τ, τ) ...	(types des n -uplets)
$\tau \rightarrow \tau$	(fonctions)
<code>bool</code>	(booléens)
<code>int</code>	(entiers)

Allons-y pour les règles d'inférence de typage. D'abord, il y en a quelques unes que nous connaissons déjà, mais qui doivent être formulées.

$$\frac{}{\Gamma, e : \tau \vdash e : \tau} \text{ (T-Ax)} \quad \frac{\Gamma, x : \alpha \vdash e : \beta}{\Gamma \vdash (\text{fun } x \rightarrow e) : \alpha \rightarrow \beta} \text{ (T-Abs)} \quad \frac{\Gamma \vdash f : \alpha \rightarrow \beta \quad \Gamma \vdash a : \alpha}{\Gamma \vdash f a : \beta} \text{ (T-App)}$$

Les constantes dans $\mathbb{Z}$ et les booléens ont toujours leur type respectif :

$$\frac{}{\Gamma \vdash n : \text{int}} \text{ (T-INT)} \quad \frac{}{\Gamma \vdash \text{true} : \text{bool}} \text{ (T-TRUE)} \quad \frac{}{\Gamma \vdash \text{false} : \text{bool}} \text{ (T-FALSE)}$$

On a aussi déjà vu la règle qui permet de typer un test booléen :

$$\frac{\Gamma \vdash g : \text{bool} \quad \Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash (\text{if } g \text{ then } e_1 \text{ else } e_2) : \tau} \text{ (T-If)}$$

Nous connaissons celles de n -uplets :

$$\forall n \in \mathbb{N} \quad \frac{\Gamma \vdash a_1 : \alpha_1 \quad \dots \quad \Gamma \vdash a_n : \alpha_n}{\Gamma \vdash (a_1, \dots, a_n) : \alpha_1 \times \dots \times \alpha_n} \text{ (T-TUPLE}_n\text{)}$$

Les règles pour les n variants (Var_i of τ_i) d'un type somme Σ sont déjà connues :

$$\forall i \in \llbracket 1, n \rrbracket \quad \frac{\Gamma \vdash e : \tau_i}{\Gamma \vdash \text{Var}_i e : \Sigma} \text{ (T-Var}_i\text{)}$$

Enfin, nous connaissons les opérations binaires sur les booléens et les entiers :

$$\frac{\Gamma \vdash a : \text{bool} \quad \Gamma \vdash b : \text{bool}}{\Gamma \vdash a \diamond_{\text{bool}} b : \text{bool}} \text{ (T-}\diamond_{\text{bool}}\text{)} \quad \frac{\Gamma \vdash a : \text{int} \quad \Gamma \vdash b : \text{int}}{\Gamma \vdash a \diamond_{\text{int}} b : \text{int}} \text{ (T-}\diamond_{\text{int}}\text{)}$$

$$\frac{\Gamma \vdash x : \text{bool}}{\Gamma \vdash *_{\text{bool}} x : \text{bool}} \text{ (T-*}_{\text{bool}}\text{)} \quad \frac{\Gamma \vdash x : \text{int}}{\Gamma \vdash *_{\text{int}} x : \text{int}} \text{ (T-*}_{\text{int}}\text{)}$$

En ce qui concerne les opérations binaires, il reste à typer les opérations de comparaison sur les types quelconques. Ces opérateurs fonctionnent sur n'importe quel type, parce qu'OCaml utilise l'égalité structurelle et l'ordre lexicographique pour comparer deux objets **du même type**. Il semble clair que le résultat d'une comparaison soit de type `bool`. Il faut par ailleurs imposer que les deux expressions en argument soient du même type. Ainsi, pour toute opération de comparaison $\circ$, on a :

$$\frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 \circ e_2 : \text{bool}} \text{ (T-}\circ\text{)}$$

Il reste trois expressions à typer. Tout d'abord, le séquençage $e_1 ; e_2$ est une opération simple. Il consiste à évaluer le premier terme e_1 qui doit être de type $()$, c'est-à-dire, le tuple vide, puis renvoie le résultat de e_2 , qui est d'un type quelconque. Ainsi, la règle de typage du séquençage est :

$$\frac{\Gamma \vdash e_1 : () \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 ; e_2 : \tau} \text{ (T-SEQ)}$$

Un let-binding (une déclaration) s'écrivant **let** $p = e_1$ **in** e_2 consiste à évaluer la première expression e_1 , puis de l'assigner au motif p avant de continuer l'évaluation de e_2 , de type quelconque. Pour pouvoir déconstruire e_1 sur p , il faut que cette expression et le motif soient du même type. Le type renvoyé par e_2 est le type de toute l'expression. D'où la règle d'inférence :

$$\frac{\Gamma \vdash p : \epsilon \quad \Gamma \vdash e_1 : \epsilon \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash (\text{let } p = e_1 \text{ in } e_2) : \tau} \text{ (T-LET)}$$

Enfin, le même principe s'applique pour le filtrage. On a une expression s qui est déconstruite sur l'un des motifs du corps de la structure **match**, donc les motifs p_i et l'expression s doivent être du même type ϵ . La valeur de retour doit être la même dans toutes les branches, donc tous les e_i sont du même type τ , qui est lui-même le type de toute l'expression.

$$\frac{\Gamma \vdash s : \epsilon \quad \Gamma \vdash p_1 : \epsilon \quad \dots \quad \Gamma \vdash p_n : \epsilon \quad \Gamma \vdash e_1 : \tau \quad \dots \quad \Gamma \vdash e_n : \tau}{\Gamma \vdash (\text{match } s \text{ with } | p_1 \rightarrow e_1 | \dots | p_n \rightarrow e_n) : \tau} \text{ (T-MATCH)}$$

Une dernière chose : nous avons parlé du typage de motif, mais nous n'avons pas écrit les règles pour leur syntaxe. Ce n'est en réalité pas grave, car leur règles sont identiques. Les voici quand même.

D'abord, la règle « axiome » pour les motifs est la même.

$$\overline{\Gamma, p : \tau \vdash p : \tau} \text{ (T-PAT-AX)}$$

Les motifs constant entiers ou booléens ont toujours le même type.

$$\overline{\Gamma \vdash n : \text{int}} \text{ (T-PAT-INT)}$$

$$\overline{\Gamma \vdash \text{true} : \text{bool}} \text{ (T-PAT-TRUE)} \quad \overline{\Gamma \vdash \text{false} : \text{bool}} \text{ (T-PAT-FALSE)}$$

Ensuite, les motifs des tuples sont aussi les mêmes :

$$\forall n \in \mathbb{N} \quad \frac{\Gamma \vdash p_1 : \alpha_1 \quad \dots \quad \Gamma \vdash p_n : \alpha_n}{\Gamma \vdash (p_1, \dots, p_n) : \alpha_1 \times \dots \times \alpha_n} \text{ (T-PAT-TUPLE}_n\text{)}$$

Les variants appelés avec des motifs sont aussis typés avec la même règle.

$$\forall i \in \llbracket 1, n \rrbracket \quad \frac{\Gamma \vdash p : \tau_i}{\Gamma \vdash \text{Var}_i p : \Sigma} \text{ (T-PAT-Var}_i\text{)}$$

T.S.V.P.

Définition Règles d'inférence pour le typage en OCaml

Expressions e :

$$\begin{array}{c}
 \frac{}{\Gamma, e : \tau \vdash e : \tau} \text{ (T-Ax)} \quad \frac{\Gamma, x : \alpha \vdash e : \beta}{\Gamma \vdash (\text{fun } x \rightarrow e) : \alpha \rightarrow \beta} \text{ (T-Abs)} \quad \frac{\Gamma \vdash f : \alpha \rightarrow \beta \quad \Gamma \vdash a : \alpha}{\Gamma \vdash f a : \beta} \text{ (T-App)} \\
 \\
 \frac{}{\Gamma \vdash n : \text{int}} \text{ (T-INT)} \quad \frac{}{\Gamma \vdash \text{true} : \text{bool}} \text{ (T-TRUE)} \quad \frac{}{\Gamma \vdash \text{false} : \text{bool}} \text{ (T-FALSE)} \\
 \\
 \frac{\Gamma \vdash g : \text{bool} \quad \Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash (\text{if } g \text{ then } e_1 \text{ else } e_2) : \tau} \text{ (T-If)} \\
 \\
 \forall n \in \mathbb{N} \quad \frac{\Gamma \vdash a_1 : \alpha_1 \quad \dots \quad \Gamma \vdash a_n : \alpha_n}{\Gamma \vdash (a_1, \dots, a_n) : \alpha_1 \times \dots \times \alpha_n} \text{ (T-TUPLE}_n\text{)} \\
 \\
 \forall i \in \llbracket 1, n \rrbracket \quad \frac{\Gamma \vdash e : \tau_i}{\Gamma \vdash \text{Var}_i e : \Sigma} \text{ (T-Var}_i\text{)} \\
 \\
 \frac{\Gamma \vdash a : \text{bool} \quad \Gamma \vdash b : \text{bool}}{\Gamma \vdash a \diamond_{\text{bool}} b : \text{bool}} \text{ (T-}\diamond_{\text{bool}}\text{)} \quad \frac{\Gamma \vdash a : \text{int} \quad \Gamma \vdash b : \text{int}}{\Gamma \vdash a \diamond_{\text{int}} b : \text{int}} \text{ (T-}\diamond_{\text{int}}\text{)} \\
 \\
 \frac{\Gamma \vdash x : \text{bool}}{\Gamma \vdash *_\text{bool} x : \text{bool}} \text{ (T-*}_{\text{bool}}\text{)} \quad \frac{\Gamma \vdash x : \text{int}}{\Gamma \vdash *_\text{int} x : \text{int}} \text{ (T-*}_{\text{int}}\text{)} \\
 \\
 \frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 \circ e_2 : \text{bool}} \text{ (T-}\circ\text{)} \\
 \\
 \frac{\Gamma \vdash e_1 : () \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 ; e_2 : \tau} \text{ (T-SEQ)} \quad \frac{\Gamma \vdash p : \epsilon \quad \Gamma \vdash e_1 : \epsilon \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash (\text{let } p = e_1 \text{ in } e_2) : \tau} \text{ (T-LET)} \\
 \\
 \frac{\Gamma \vdash s : \epsilon \quad \Gamma \vdash p_1 : \epsilon \quad \dots \quad \Gamma \vdash p_n : \epsilon \quad \Gamma \vdash e_1 : \tau \quad \dots \quad \Gamma \vdash e_n : \tau}{\Gamma \vdash (\text{match } s \text{ with } | p_1 \rightarrow e_1 \mid \dots \mid p_n \rightarrow e_n) : \tau} \text{ (T-MATCH)}
 \end{array}$$

Motifs p :

$$\begin{array}{c}
 \frac{}{\Gamma, p : \tau \vdash p : \tau} \text{ (T-PAT-Ax)} \\
 \\
 \frac{}{\Gamma \vdash n : \text{int}} \text{ (T-PAT-INT)} \quad \frac{}{\Gamma \vdash \text{true} : \text{bool}} \text{ (T-PAT-TRUE)} \quad \frac{}{\Gamma \vdash \text{false} : \text{bool}} \text{ (T-PAT-FALSE)} \\
 \\
 \forall n \in \mathbb{N} \quad \frac{\Gamma \vdash p_1 : \alpha_1 \quad \dots \quad \Gamma \vdash p_n : \alpha_n}{\Gamma \vdash (p_1, \dots, p_n) : \alpha_1 \times \dots \times \alpha_n} \text{ (T-PAT-TUPLE}_n\text{)} \\
 \\
 \forall i \in \llbracket 1, n \rrbracket \quad \frac{\Gamma \vdash p : \tau_i}{\Gamma \vdash \text{Var}_i p : \Sigma} \text{ (T-PAT-Var}_i\text{)}
 \end{array}$$

4.2 Vue d'ensemble de l'algorithme

L'algorithme que nous allons coder prend donc en entrée une expression e (ou un motif p), et doit trouver un type τ tel que $\Gamma \vdash e : \tau$ (ou $\Gamma \vdash p : \tau$). Pour cela, il doit respecter les règles d'inférence de typage en OCaml. Plusieurs problèmes sont à considérer si l'on veut espérer mettre au point un tel algorithme.

• Gestion du contexte

Le contexte, intuitivement, est la structure qui « se souvient » des variables déclarées dans des `let`, des `match` ou des `fun`. De plus, à chaque variable déclarée, il associe un type τ tel que $\Gamma \vdash x : \tau$ (où Γ est le contexte lui-même).

On peut en premier lieu approcher de problème de manière naïve : simplement utiliser un tableau associatif. Mais il y a un autre problème que nous n'avons pas considéré. C'est ce qu'on appelle couramment en programmation le « *shadowing* ». Le shadowing est une fonctionnalité en OCaml qui permet de redéfinir une variable avec le même nom. La nouvelle variable n'a rien à voir et ne pose aucun problème, mais on n'a plus accès aux variables précédentes du même nom tant que la nouvelle variable est à l'atteinte dans la portée.

Exemple Dans le code suivant, `x` est redéfini dans une portée plus profonde :

```
let calcul x =  
  if une_condition then  
    let x = calcul ()  
    in x + 1  
  else  
    x - 1
```

Il faut alors envisager la possibilité **d'imbriquer des contextes**. On peut toujours se servir d'un tableau associatif, mais on doit aussi stocker la portée précédente dans laquelle le contexte actuel est imbriqué.

Règles ne précisant pas les types à utiliser

Dans certaines règles de typage, par exemple la règle T-APP, il faut astucieusement choisir le bon type α qui permettra de construire un arbre de dérivation terminé. Il semble difficile d'imaginer un algorithme capable de le trouver en analysant toutes les informations possibles. Il faut donc pouvoir « créer » des nouveaux types qu'on donne à des expressions (ou motifs), en espérant que l'algorithme déduise de quels types il s'agissait vraiment.

Contraintes

Dans d'autres règles, il faut vérifier que deux expressions ont le même type τ (par exemple dans T-IF). L'algorithme doit ainsi être capable de mettre en relation deux types par égalité. Mais en faisant cela, il doit vérifier qu'il ne met pas en relations deux types contenant les mêmes variables, sous risque de créer des types récursifs infinis – horreur !

Exemple On ne peut jamais avoir $\alpha = \alpha \rightarrow \beta$. Ceci impose que l'on ne peut jamais calculer une expression de la forme `f f` (appel d'une fonction sur elle-même).

Etant donné tous ceux problèmes, on envisage un algorithme « solveur » de contraintes : il parcourt les expressions récursivement, comme une induction structurelle, génère des types à déterminer, puis les met en relation d'égalité en s'appuyant sur les règles d'inférences. Il finit enfin par résoudre un système d'équations sur l'ensemble des types $\mathcal{T}$.

C'est un peu ce que fait un humain lorsqu'il type une fonction.

- On commence par donner un type inconnu aux variables considérées.
- On regarde comment agissent ces variables pour en déduire des égalités avec les types d'autres expressions.
- On calcule le type de retour de l'expression globale considérée.
- On résout le système d'équations trouvé pour en déduire le type final de l'expression.

Rapidement, faisons un exemple.

Exemple Typons le code suivant comme dans un exercice de sup.

```
let rec map f l = match l with
| [] -> []
| x :: l' -> (f x) :: map f l'
```

Créons un nouveau type frais pour les valeurs indéterminées. $f : \alpha$ et $l : \beta$.

La variable l est filtrée contre des motifs de listes, donc $\exists \gamma \in \mathcal{T} \mid \beta = \gamma \text{ list}$.

Le premier cas du filtrage renvoie une liste vide, donc la valeur de retour est $\delta \text{ list}$.

Dans le deuxième cas, on définit deux variables $x : \gamma'$ et $l' : \beta'$. Puisque le constructeur $(::)$ sur les listes est de type $\gamma \times (\gamma \text{ list}) \rightarrow (\gamma \text{ list})$, cela nous donne deux contraintes : $\begin{cases} \gamma' = \gamma \\ \beta' = \beta = \gamma \text{ list} \end{cases}$

Ensuite, la fonction f est appelée sur $x : \gamma$, donc $f : \gamma \rightarrow \epsilon = \alpha$ pour un certain $\epsilon \in \mathcal{T}$.

La fonction map est appelée récursivement avec la même valeur de f et une nouvelle liste l' . Ceci vérifie bien que $\gamma \rightarrow \epsilon = \gamma \rightarrow \epsilon$ et que $\gamma \text{ list} = \gamma \text{ list}$.

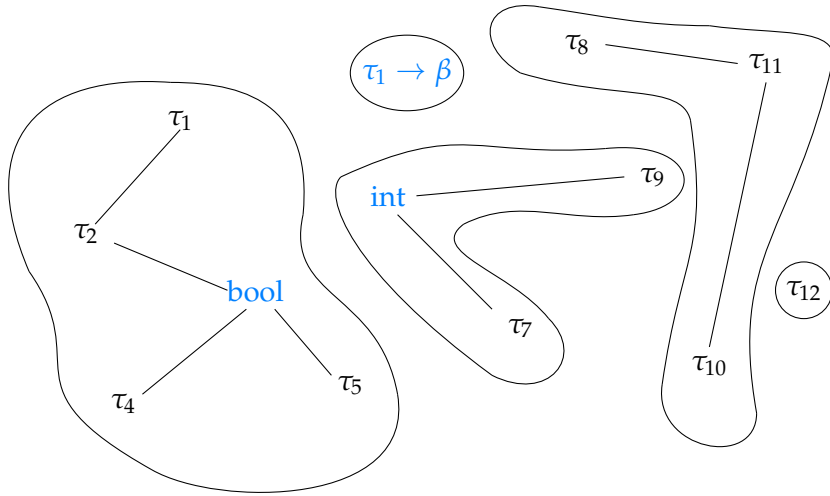
Le type retourné par le deuxième branche est celui de $(f \ x) :: \text{map } f \ l'$. Ceci impose que $\begin{cases} \epsilon = \epsilon \\ \delta \text{ list} = \epsilon \text{ list} \end{cases}$

Au final, cette fonction est donc de type

$$\underbrace{(\underbrace{\gamma \rightarrow \epsilon}_f) \rightarrow \underbrace{\gamma \text{ list}}_l}_{\text{map}} \rightarrow \epsilon \text{ list}$$

Comment résoudre les contraintes ?

On prévoit que notre algorithme va créer de nouveaux types pour chaque nouvelles variables rencontrés, et les mets en relation par égalité. Ainsi, si l'on considère l'ensemble des types créés dans le contexte Γ , et les liens d'égalité qui décrit les contraintes :



On voit que cet ensemble peut être représenté comme un graphe, où $\tau_i = \tau_j \iff \tau_i \longleftrightarrow \tau_j$. On en déduit que les composantes connexes sont les classes d'équivalences de la relation $(=)$ sur les types du contexte ! Une manière de résoudre un système d'équation est alors de fusionner les classes d'équivalences de deux types lorsqu'ils sont mis en égalité : cette tâche est facilement effectuée par la structure Union-Find.

Remarquons que si l'un des types dans une classe d'équivalence n'est pas une variable, alors ce type représente le type de la classe d'équivalence, et donc de toutes les expressions dont le type est dans la même classe.

Grâce à la structure Union-Find, on peut donc mettre à jour une partition du contexte Γ , tel que $\text{find}(\tau)$ est le représentant de cette classe. Attention, il faut imposer que $\text{find}(\tau)$ soit, dès que possible, un type qui n'est pas une variable, car le représentant sera la solution que l'on devra extraire des classes d'équivalences.

Remarque L’algorithme que nous sommes en train de décrire est l’**algorithme d’inférence de Damas-Hindley-Milner**.

Quand résoudre les contraintes ?

Notre algorithme va établir un système d’équations, mais quand va-t-il le résoudre ? Voici deux méthodes pour résoudre un système d’égalités dans le cadre de l’inférence de types.

- La première consiste à résoudre de fameux système après avoir généré tous les types et toutes les contraintes. C’est l’algorithme le plus formel, et s’appuie sur la méthode de substitution dans un système. Plus précisément, on exprime un type à partir d’autres à l’aide d’une équation, puis on enlève cette équation, et on remplace toutes les occurrences de ce type par le nouveau. On recommence jusqu’à obtenir un système résolu.

Cet algorithme s’appelle l’**algorithme W**.

- La deuxième méthode est plus « gloutonne » : elle fusionne les classes d’équivalences dès qu’une nouvelle contrainte est générée, et vérifie que les représentants des classes sont bien compatibles (en imposant une nouvelle contrainte).

Cet algorithme s’appelle l’**algorithme J**.

L’algorithme J est souvent plus simple à implémenter, et puisque nous travaillons avec la structure Union-Find, c’est celui-ci que nous allons utiliser.

Remarque L’algorithme J s’exécute en $\mathcal{O}(n\alpha(n))$, où n est le nombre total de types dans le contexte Γ .

Rappel : α est la fonction inverse d’Ackermann ($n \mapsto A(n, n)$), et grandit extrêmement lentement. En pratique, on ne dépassera jamais $\alpha(n) > 4$, ce qui rend la complexité quasi-linéaire en n .

En effet, $A(4, 4) \approx 2^{2^{2^{16}}}$.

4.3 Algorithme J : une implémentation

L’algorithme J est ici partiellement implémenté en OCaml.

4.3.1 Préliminaires

On va définir les structures que l’on manipule. Notamment, les expressions, les motifs, les types, etc. Pour chacune d’entre elles, on s’appuie sur la définition de la partie (4.1).

```
type expr =
| EVar of var
| EFun of pat * expr
| EApp of expr * expr
| ELet of (pat * expr) * expr
| ESeq of expr * expr
| ETuple of expr array
| EVariant of int * expr
| EMatch of expr * (pat * expr) array
| EBool of bool
| EIf of expr * expr * expr
| EAnd of expr * expr | EOr of expr * expr | ENot of expr
| EInt of int
| EAdd of expr * expr | ESub of expr * expr | EMul of expr * expr | EDiv of expr * expr
| ENeg of expr
| EEq of expr * expr | ENe of expr * expr
| ELe of expr * expr | ELt of expr * expr
| EGe of expr * expr | EGt of expr * expr
```

```

and pat =
| PVar of var
| PTuple of pat array
| PVariant of int * pat
| PBool of bool
| PInt of int
and var = int
and ty_id = int
and ty =
| TVar of ty_id
| TTuple of ty_id array
| TFun of ty_id * ty_id
| TBool | TInt

```

Remarque Je laisse intentionnellement un niveau d'indirection supplémentaires pour la structure des types, car quand j'utiliserai la structure Union-Find sur les types, je vais avoir besoin de faire références aux classes d'équivalences d'un type, et j'aurai donc besoin de son indice dans la structure U-F.

4.3.2 Contexte

Rappelons-nous que le contexte doit être un tableau associatif, et un pointeur vers un contexte précédemment pour prendre en compte le shadowing. Je propose la structure suivante :

```

type ctx = {
  variables : (var, ty) Hashtbl.t ;
  prev : ctx option ;
}

```

Le premier contexte créé n'a pas de contexte parent, donc on l'initialise avec la fonction suivante :

```

let ctx_init () = {
  variables = Hashtbl.create 0;
  prev = None;
}

```

La fonction suivante permet d'ajouter une nouvelle variable dans le contexte actuel par effet de bord. Cette action risque d'assombrir (shadow) une variable du même nom qui était déjà dans le contexte !

```

let declare_variable ctx (var, ty) =
  Hashtbl.replace ctx.variables var ty

```

Enfin, la fonction suivante cherche le type d'une variable dans le contexte étant donné son identifiant. Il faut faire une recherche récursive :

- Si la variable se trouve dans la portée actuelle, alors on renvoie son type.
- Sinon, on cherche dans la portée précédente.
- Si le contexte actuel est la racine, on émet une erreur : « Variable inconnue ».

```

exception Unknown_Variable of var
let rec find_variable ctx x =
  match Hashtbl.find_opt ctx.variables x with
  | Some ty -> ty
  | None -> match ctx.prev with
    | Some ctx' -> find_variable ctx' x
    | None -> raise (Unknown_Variable x)

```

Cela devrait être bon pour le contexte.

4.3.3 Union-Find

Je ne vais pas détailler l'implémentation d'une structure Union-Find ici, mais voici sa signature :

```
type 'a uf
uf_create: () -> 'a uf
uf_add: 'a uf -> 'a -> int
uf_find: 'a uf -> int -> int
uf_union: 'a uf -> int -> int -> unit
uf_get: 'a uf -> int -> 'a
```

Vous remarquerez peut-être qu'elle est un peu différente de ce qu'on a l'habitude d'utiliser. Oui, en effet. J'ai modifié la structure pour qu'elle stocke des valeurs de type 'a, ce qui me permettra de récupérer la valeur du représentant d'une classe d'équivalence.

4.3.4 Unification des contraintes

Pour résoudre le système, il faut que l'on fusionne deux classes d'équivalences si une égalité porte sur deux types inconnus. S'il les deux sont connus (i.e. ce ne sont pas des variables), on vérifie qu'ils sont bien égaux en regardant s'ils ont bien la même forme, et en ajoutant des contraintes supplémentaires sur leur types intérieurs.

Cette étape s'appelle *l'unification*.

En détail, l'algorithme se déroule de la manière suivante :

- **Entrée** : deux types α' et β' (représentés par un `ty_id`, leur position dans la structure U-F).
 - On récupère leur type représentant α et β , car ils sont égaux, et on veut raisonner sur les représentants des classes d'équivalence.
 - Cas des variables : l'un d'entre eux est une variable. Sans perte de généralité, on suppose que α et x .
 - **On vérifie que $\alpha = x$ n'apparaît pas dans l'écriture de β !!** On ne peut pas autoriser les types récursifs, donc si jamais c'est le cas, on émet une erreur : « Erreur de typage (type infini), $x \neq \beta$ ».
 - Si l'étape précédente a été vérifiée, alors on impose que $x = \beta$ en fusionnant les classes de α et β . En le faisant, **on s'assure que le nouveau représentant de cette classe soit β** , c'est-à-dire, après fusion, $\text{find}(\alpha) = \text{find}(\beta) = \beta$. On a déjà expliqué pourquoi ceci devait être le cas, c'est parce qu'on privilégie des types concrets plutôt que des variables en tant que représentants.
 - $\alpha = \text{int}$ et $\beta = \text{int}$. Il n'y a rien à vérifier d'autre, donc on retourne `unit`.
 - $\alpha = \text{bool}$ et $\beta = \text{bool}$. Idem.
 - $\alpha = \tau_1 \rightarrow \tau_2$ et $\beta = \mu_1 \rightarrow \mu_2$. On impose que $\tau_1 = \mu_1$ et que $\tau_2 = \mu_2$ en appelant récursivement l'algorithme d'unification sur l'entrée (τ_1, μ_1) puis (τ_2, μ_2) .
 - $\alpha = (\alpha_1, \dots, \alpha_n)$ et $\beta = (\beta_1, \dots, \beta_m)$. D'une part, on vérifie manuellement que $m = n$, sinon erreur ! « Erreur de typage (taille de tuples incompatibles) ».
- Ensuite, $\forall i \in \llbracket 1, n \rrbracket$, on impose que $\alpha_i = \beta_i$ avec un appel récursif à l'unification.
- Autre cas : les types ne peuvent pas être égaux, donc on émet immédiatement une erreur. « Erreur de typage : $\alpha \neq \beta$ ».

Tout d'abord, définissons une structure qui va contenir les informations utiles à l'inférence de type (le contexte et la structure U-F).

```
type checker = {
  types: ty uf ;
  mutable context: ctx;
}
```

Ensuite, écrit la fonction qui vérifie si une variable x apparaît dans l'écriture d'un type τ .

On l'appelle `occurs_in`: `checker -> ty_id -> ty_id -> bool`

```
let rec occurs_in checker x tau =
  match uf_get checker.types tau with
  | TVar y -> x = y
  | TBool | TInt -> false
  | TFun (a, b) -> occurs_in checker x a || occurs_in checker x b
  | TTuple (tys) -> Array.exists (occurs_in checker x) tys
```

La fonction fait une disjonction de cas sur τ , et si $\tau = y$, alors on vérifie que $x = y$, sinon, on effectue des appels récursivement en regardant si au moins l'un d'entre eux renvoie true.

La fonction `unify`: `checker -> ty_id -> ty_id -> unit` s'écrit alors :

```
exception Occurs_In_Fail of ty_id * ty_id
exception Type_Error of ty_id * ty_id
let rec unify checker a b =
  match (uf_get checker.types a, uf_get checker.types b) with
  | (TVar xa, _) -> if occurs_in checker xa b
    then raise (Occurs_In_Fail (a, b))
    else uf_union checker.types a b
  | (_, TVar xb) -> if occurs_in checker xb a
    then raise (Occurs_In_Fail (b, a))
    else uf_union checker.types b a
  | (TInt, TInt) -> ()
  | (TBool, TBool) -> ()
  | (TFun (t1, t2), TFun (u1, u2)) ->
    unify checker t1 u1;
    unify checker t2 u2;
  | (TTuple aa, TTuple bb) when Array.length aa = Array.length bb ->
    let n = Array.length aa in
    for i = 0 to n - 1 do
      unify checker aa.(i) bb.(i)
    done
  | _ -> raise (Type_Error (a, b))
```

4.3.5 Inférence

Presque terminé, il ne reste plus qu'à générer des contraintes en parcourant les expressions et les motifs dont on veut le type. On fait cela en générant des appels à `unify`, qui s'occupe des classes d'équivalences.

Comment générer les contraintes ?

Il faut s'appuyer sur les règles d'inférences. Commençons par les motifs :

- **Entrée** : un motif p . **Sortie** : son type τ .
- Par disjonction de cas sur la forme de p , qui correspond à une règle d'inférence ((T-PAT)-...).
- $p = x$ une variable : règle (T-PAT-AX). On commence par créer un tout nouveau type α qui sera le type de x . On ajoute $(x : \alpha)$ au contexte Γ , puis on renvoie simplement α .
- $p = \text{true}$ ou $p = \text{false}$: règle (T-PAT-TRUE) ou (T-PAT-FALSE). On renvoie directement le type `bool`.
- $p = n \in \mathbb{Z}$: règle (T-PAT-INT). On renvoie directement le type `int`.
- $p = (p_1, \dots, p_n)$: règle (T-PAT-TUPLE _{n}). On infère récursivement les motifs $p_1, \dots, p_n$, ce qui nous donne les types $\tau_1, \dots, \tau_n$, et on renvoie directement le type tuple formé de ces derniers : $(\tau_1, \dots, \tau_n)$.
- $p = \text{Var}_i p'$: règle (T-PAT-Var _{i}). On récupère les type τ_i associé au variant Var_i de Σ . On procède récursivement en inférant le type τ' de p' , puis on vérifie $\tau_i = \tau'$ par unification. Enfin, on renvoie τ_i .

Je ne détaille pas comment j'obtiens le type d'un variant Var_i (cela dépend en vérité de la structure de votre code), et je me donne une fonction magique qui le fait à ma place : `get_variant : int -> ty_id`.

La fonction `infer_pattern : checker -> pat -> ty_id` qui infère le type d'un motif peut être écrite comme ceci :

```
let rec infer_pattern checker p =
  match p with
  | PVar x ->
    let alpha = create_fresh_type checker in
    declare_variable checker.context (x, alpha);
    alpha
  | PBool _ -> create_type checker TBool
  | PInt _ -> create_type checker TInt
  | PTuple pp ->
    create_type checker
      (TTuple (Array.map (infer_pattern checker) pp))
  | PVariant (i, p') ->
    let tau_i = get_variant i in
    let tau' = infer_pattern checker p' in
    unify checker tau_i tau';
    tau_i
```

Même raisonnement pour les expressions, sauf qu'il y a beaucoup plus de cas !

- **Entrée** : une expression e . **Sortie** : son type τ .
- Par disjonction de cas sur la forme de e , qui correspond à une règle d'inférence ((T)-...).
 - $e = x$ une variable : règle (T-AX). On récupère le type de x en fouillant dans le contexte, puis on le retourne. Si la variable n'existe pas, ceci émet une erreur.
 - $e = \text{fun } p \rightarrow e'$: règle (T-ABS). Déjà, avant toute chose, on ouvre un nouveau contexte, pour empêcher les variables de la fonction d'en sortir. Dans ce contexte, on infère le motif $p : \tau_1$, et *seulement après* on infère le corps de la fonction $e' : \tau_2$. On ferme le contexte puis on renvoie le type $\tau_1 \rightarrow \tau_2$.
 - $e = f e'$: règle (T-APP). On commence par inférer le type de $f : \phi'$. Ensuite, on infère le type de l'argument $e' : \alpha$, et on crée un nouveau type de retour β . On pose $\phi = \alpha \rightarrow \beta$, et on impose par unification que $\phi = \phi'$. Le type de retour de l'appel est β d'après la règle d'inférence, donc c'est le type que l'on renvoie.
 - $e = \text{let } p = e' \text{ in } e''$: règle (T-LET). Il faut que e' puisse rentrer dans p d'après la règle. On infère donc $p : \epsilon$ puis $e' : \epsilon'$, et par unification, on impose $\epsilon = \epsilon'$. Il ne reste qu'à renvoyer le type de $e'' : \tau$, que l'on infère récursivement.
 - $e = e' ; e''$: règle (T-SEQ). On impose par unification que le type inféré de $e : \tau'$ soit $\tau' = ()$, puis on renvoie l'inférence du type de e'' immédiatement.
 - $e = \text{true}$ ou $p = \text{false}$: règle (T-TRUE) ou (T-FALSE). Idem que pour les motifs `true` ou `false`.
 - $e = \text{if } g \text{ then } a \text{ else } b$: règle (T-IF). On impose par unification que le type inféré de $g : \gamma$ soit $\gamma = \text{bool}$. Ensuite, on infère les types des deux valeurs possibles, $a : \tau_a$ et $b : \tau_b$, puis on impose par unification que $\tau_a = \tau_b = \tau$. On renvoie τ .
 - $e = n \in \mathbb{Z}$: règle (T-INT). Idem que pour le motif entier.
 - $e = (p_1, \dots, p_n)$: règle (T-TUPLE_n). Idem que pour le motif n -uplet.
 - $e = \text{Var}_i p'$: règle (T--Var_i). Idem que pour le motif du variant Var_i .
 - $e = \text{match } s \text{ with } | p_1 \rightarrow e_1 | \dots | p_n \rightarrow e_n$: règle (T-MATCH). Inférons d'abord le type de $s : \sigma$, puis créons un nouveau type frais τ . Pour chaque cas $i \in \llbracket 1, n \rrbracket$, on infère les types $e_i : \tau_i$ et $p_i : \sigma_i$. D'après la règle, tous les motifs doivent être du type σ , donc on impose par unification que $\sigma_i = \sigma$. De même, toutes les valeurs de retour doivent être τ , donc par unification on impose $\tau_i = \tau$. Il ne reste plus qu'à renvoyer τ .

- Dans le cas où e est une opération sur les booléens, les entiers, ou une opération de comparaison
On applique la règle respective (T- $\diamond$). Il faut que les types inférés de ou des arguments soient égaux (imposé par unification). Si l'opération est sur les entiers, on impose en plus que le ou les arguments soient entiers par unification. Idem pour les booléens. On renvoie le type associé au résultat de l'opération : c'est un entier si c'est une opération arithmétiques, un booléen si c'est une opération booléenne, et sinon un booléen si c'est un test.

Wouah, nous y sommes ! La fonction codée en OCaml, `infer : checker -> expr -> ty_id`, est :

```
let rec infer checker e =
  match e with
  | EVar x -> find_variable checker.context x
  | EFun (p, e') ->
    open_scope checker;
    let arg = infer_pattern checker p in
    let ret = infer checker e' in
    close_scope checker;
    create_type checker (TFun (arg, ret))
  | EApp (f, e') ->
    let phi' = infer checker f in
    let arg = infer checker e' in
    let ret = create_fresh_type checker in
    let phi = create_type checker (TFun (arg, ret)) in
    unify checker phi phi';
    ret
  | ELet ((p, e'), e'') ->
    let eps = infer_pattern checker p in
    let eps' = infer checker e' in
    unify checker eps eps';
    infer checker e''
  | ESeq (e', e'') ->
    let unit = create_type checker (TTuple [||]) in
    unify checker unit (infer checker e');
    infer checker e''
  | EBool _ -> create_type checker TBool
  | EIf (g, a, b) ->
    let bool = create_type checker TBool in
    unify checker bool (infer checker g);
    let tau_a = infer checker a in
    let tau_b = infer checker b in
    unify checker tau_a tau_b;
    tau_a
  | EInt _ -> create_type checker TInt
  | ETuple ee ->
    create_type checker
      (TTuple (Array.map (infer checker) ee))
  | EVariant (i, p') ->
    let tau_i = get_variant i in
    let tau' = infer checker p' in
    unify checker tau_i tau';
    tau_i
  | EMatch (s, cases) ->
    let sigma = infer checker s in
    let tau = create_fresh_type checker in
    Array.iter (fun (p, e) ->
      let sigma' = infer_pattern checker p in
      let tau' = infer checker e in
      unify checker sigma sigma';
      unify checker tau tau';
    ) cases; tau
```

```

| EAdd (a, b) | ESub (a, b) | EMul (a, b) | EDiv (a, b) ->
  let int = create_type checker TInt in
  unify checker int (infer checker a);
  unify checker int (infer checker b);
  int
| ENeg e' ->
  let int = create_type checker TInt in
  unify checker int (infer checker e');
  int
| EAnd (a, b) | EOr (a, b) ->
  let bool = create_type checker TBool in
  unify checker bool (infer checker a);
  unify checker bool (infer checker b);
  bool
| ENot e' ->
  let bool = create_type checker TBool in
  unify checker bool (infer checker e');
  bool
| EEq (a, b) | ENe (a, b)
| ELe (a, b) | ELt (a, b)
| EGe (a, b) | EGt (a, b) ->
  let tau = create_fresh_type checker in
  unify checker tau (infer checker a);
  unify checker tau (infer checker b);
  create_type checker TBool

```

Bravo !

5 Extensions et autres systèmes de typage

5.1 Polymorphisme, types quantifiés

5.2 Sous-typage

5.3 Contraintes sémantiques

Bibliographie

- BENJAMIN C. PIERCE, **Types and Programming Languages**
- YVES BERTOT, **Introduction au lambda-calcul pur** – issu d'un ADS de l'X.